SEMANA 9 · IDENTIDAD AVANZADA

# Roles y credenciales temporales

Parte 1 de 3 · Unidad 2

**Ing. Rodolfo Cañas Cervantes**

Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semana 9 · Periodo 2026-2 · Barranquilla, Colombia

Cómo dejar de repartir llaves: STS, assume role, perfiles de instancia y acceso entre cuentas.

ENCUADRE

# De dónde venimos y hacia dónde vamos

Las semanas 2 y 5 pusieron los cimientos; hoy aprendes a operarlos a escala, sin llaves y sin permisos de sobra.

## Ya lo dominas (semanas 2 y 5)



**Usuario raíz y MFA**
Se protege y se guarda



**Usuarios y grupos**
Personas con permisos



**Políticas JSON**
Allow y Deny



**Roles**
Identidad sin contraseña



**Evaluación**
Deny explícito gana

*Pero un entorno real tiene cientos de identidades, varias cuentas y presión por costos.*

## Hoy vamos más allá: cuatro peldaños

PARTE 1

### Roles y credenciales temporales

STS, assume role, perfiles de instancia, cross-account

PARTE 2

### Políticas a fondo

Condiciones, límites de permisos, Access Analyzer

PARTE 3

### Personas y aplicaciones

Federación, SSO, Cognito, secretos y claves

PARTE 4

### Multi-cuenta y costos

Organizations, SCP, etiquetas, presupuestos y TCO

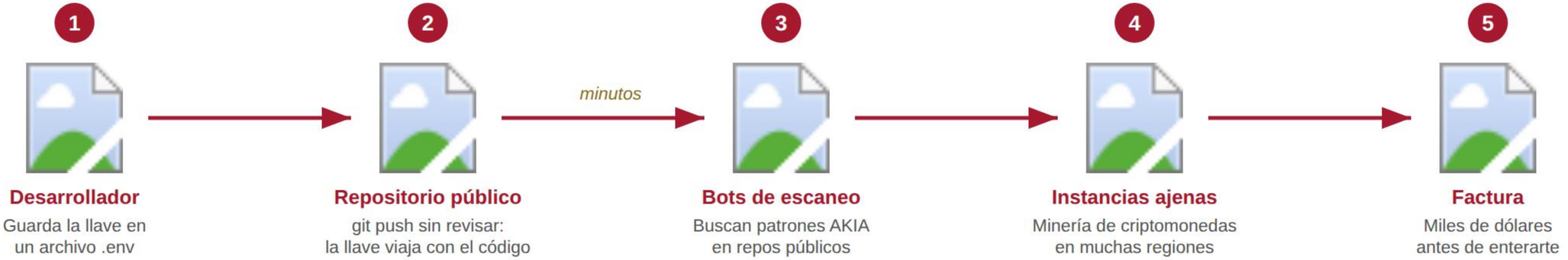


*Fuente: adaptado de AWS IAM User Guide y del temario del curso (S02, S05)*

EL PROBLEMA

# Una llave filtrada: la puerta que nadie cierra

Una llave de largo plazo en un repositorio es una puerta abierta que nadie recuerda haber dejado así.




```
# archivo .env (nunca debería existir)
AWS_ACCESS_KEY_ID=AKIAEXAMPLE0KEY123
AWS_SECRET_ACCESS_KEY=wJal...EXAMPLE
```

**Por qué es tan grave**

La llave no vence, no sabe quién la usa y funciona desde cualquier lugar del mundo con los mismos permisos que tenía su dueño.

## Cómo se evita

**1 Sin llave: usa un rol o SSO**

**2 Si existe: mínimo permiso y rotación**

**3 Detecta: último uso y alertas**



Dato de contexto: los prefijos delatan el tipo. AKIA es una llave de largo plazo de un usuario IAM; ASIA es una credencial temporal entregada por STS. Cuando veas AKIA en un repositorio, actúa de inmediato.



*Fuente: adaptado de AWS IAM Best Practices (credenciales temporales) y casos públicos de filtración*

CONCEPTO

# Credenciales de largo plazo frente a temporales

Cambiar una llave eterna por una sesión que caduca reduce el daño de cualquier filtración.

## Ventana de exposición si la credencial se filtra (escala ilustrativa)

**Llave IAM de largo plazo (AKIA)**

**No vence: meses o años hasta que alguien la elimine**

**Credencial temporal, tope de 12 h**

**Hasta 12 h**

**Credencial temporal, 1 h (por defecto)**

**1 h**

**Credencial temporal, mínimo**

**15 min**

*Cada franja dorada es una puerta que se cierra sola; la franja vino sigue abierta hasta que alguien la elimine.*





| | Largo plazo | Temporal |
|---|---|---|
| Vive | en disco, scripts, repos | en memoria, se renueva |
| Vence | nunca | 15 min a 12 h |
| Rotación | manual, la olvidan | automática |
| Si se filtra | sirve hasta revocarla | caduca sola |



**Regla práctica del curso**

Las personas usan SSO, las máquinas usan roles, los pipelines usan OIDC. Una llave de largo plazo solo se acepta cuando el sistema que llama no soporta nada mejor, y entonces se rota, se limita y se vigila.



*Fuente: adaptado de AWS IAM User Guide — Temporary security credentials in IAM*

# Un rol tiene dos políticas: confianza y permisos

La política de confianza decide quién entra; la de permisos decide qué puede hacer quien entró.

**Quién puede ser de confianza (Principal)**

- **Usuario o rol IAM**
- **Servicio AWS (ec2.amazonaws.com)**
- **Otra cuenta AWS**
- **Identidad federada (SAML u OIDC)**

### 1 Política de confianza

responde: ¿quién puede asumirme?

```
{
  "Effect": "Allow",
  "Principal": {"AWS":
   "arn:aws:iam::111111111111:root"},
  "Action": "sts:AssumeRole"
}
```



**Rol CIC-LectorReportes**

sin contraseña, sin llaves propias

### 2 Política de permisos

responde: ¿qué puedo hacer?

```
{
  "Effect": "Allow",
  "Action": "s3:GetObject",
  "Resource":
   "arn:aws:s3:::cic-reportes/*"
}
```



cic-reportes

**Idea central**

Un rol no es una persona: es un conjunto de permisos que alguien asume por un rato.

Quien lo asume recibe credenciales temporales y trabaja con los permisos del rol, no con los suyos. Para que funcione deben estar de acuerdo las dos políticas:

**confianza para entrar, permisos para actuar.**

*Los números de cuenta del curso son de ejemplo.*

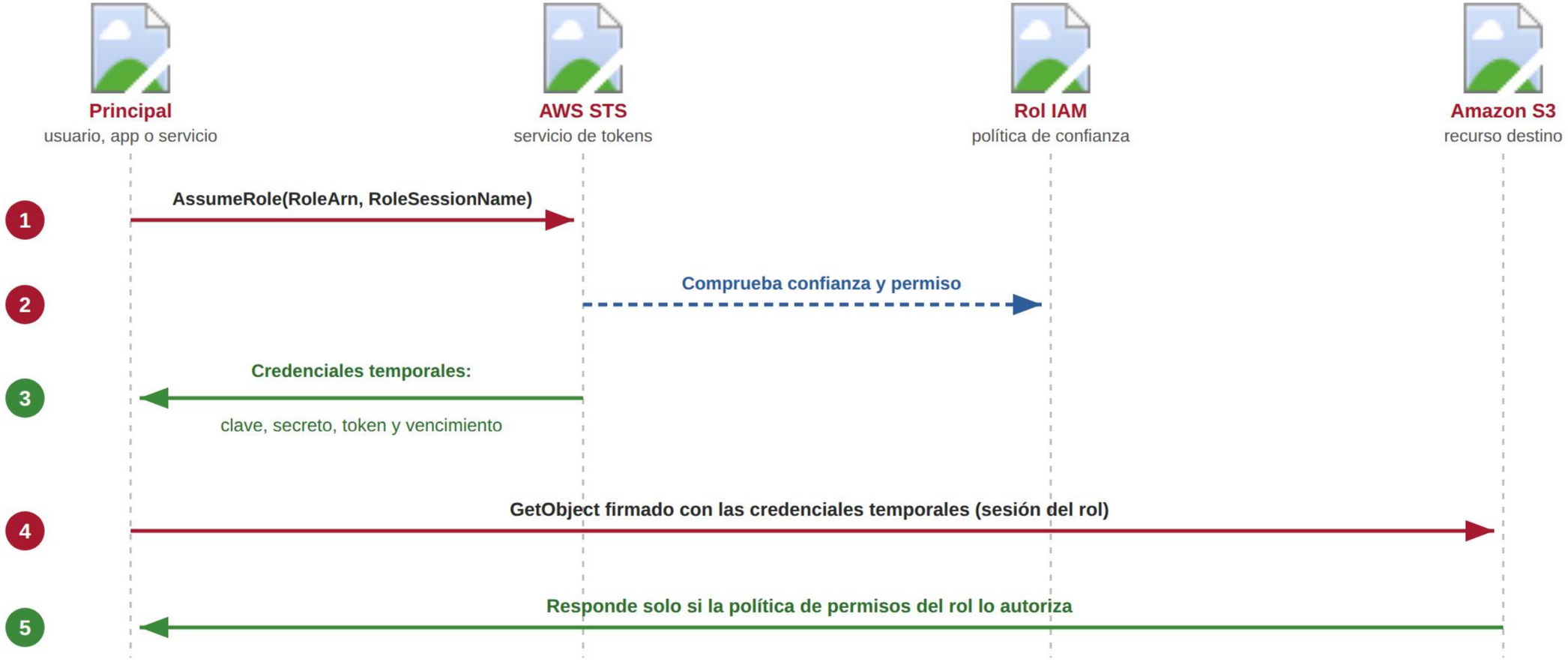


*Fuente: adaptado de AWS IAM User Guide — IAM roles y trust policies*

FLUJO PASO A PASO

# AssumeRole con STS: de la petición a la sesión

El servicio de tokens verifica la confianza y entrega credenciales que caducan solas.




Al llegar el vencimiento el principal repite el paso 1. No hay nada que revocar ni nada que rotar a mano. El paso 2 exige dos cosas a la vez: confianza en el rol y permiso sts:AssumeRole para quien llama.



*Fuente: adaptado de AWS IAM User Guide — AssumeRole y AWS STS API Reference*

ANATOMÍA

# Qué trae una sesión temporal y cuánto dura

Cuatro piezas y un reloj: la duración la fija el rol, entre 15 minutos y 12 horas.

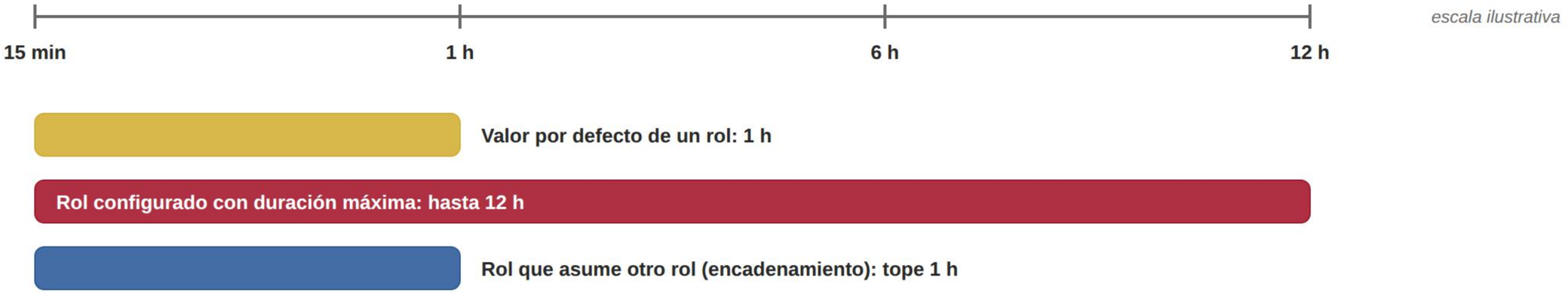
## Una sesión temporal tiene cuatro piezas

```
AccessKeyId      ASIAEXAMPLE0SESSION
SecretAccessKey  wJalr...EXAMPLEKEY
SessionToken     IQoJb3JpZ2luX2VjE...
Expiration       2026-10-04T15:00:00Z
```



**Las cuatro juntas**
o ninguna sirve

## ¿Cuánto dura la sesión?



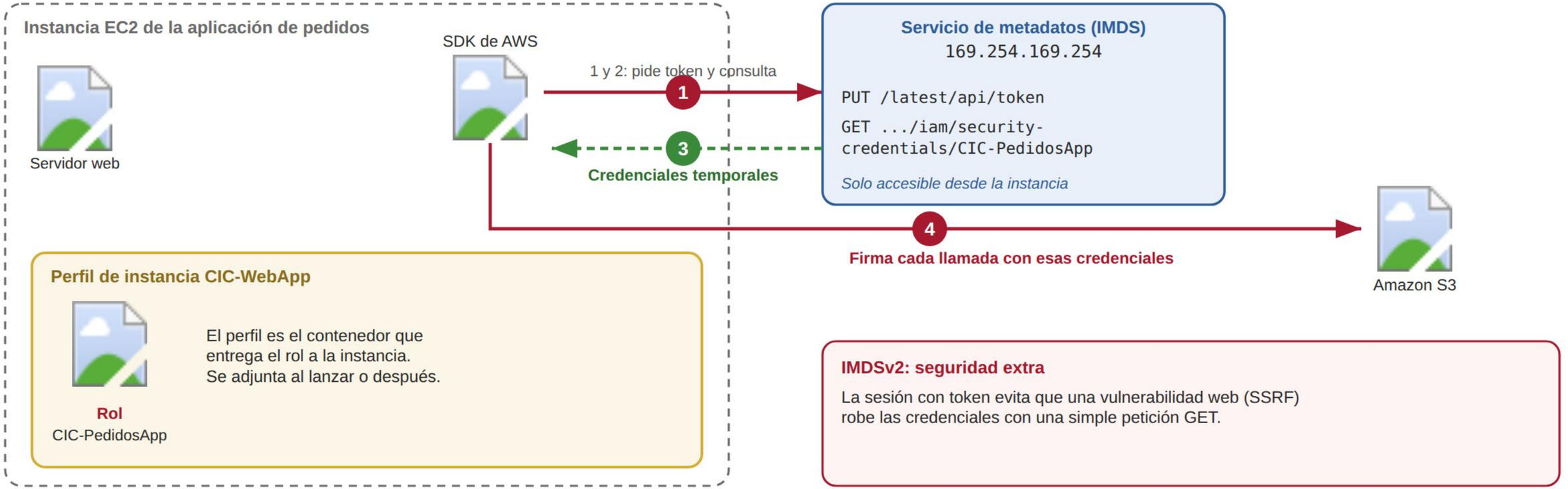

*Los servicios que usan roles (EC2, Lambda, ECS) renuevan las credenciales por ti antes de que venzan.*



*Fuente: adaptado de AWS IAM User Guide — Temporary credentials y duración de sesión de rol*

# Perfiles de instancia: el rol de una EC2

La instancia pide sus credenciales al servicio de metadatos y el SDK las renueva sin intervención.




**Qué se gana**

Cero llaves en el disco ni en el repositorio. Las credenciales se renuevan solas antes de vencer.

El permiso es por rol de aplicación, no por persona. Si borras el rol, borras el acceso, en un solo lugar.



*Fuente: adaptado de AWS EC2 User Guide — IAM roles for Amazon EC2 e IMDSv2*

# Roles de servicio: cuando AWS actúa en tu nombre

Cada servicio asume un rol propio; así los permisos viven con la carga de trabajo y no con una persona.




Lo asume la función cada vez que corre. Tú escribes los permisos.

**Permiso mínimo: solo la tabla que necesita.**




Lo asumen los contenedores de la tarea, no el host que los ejecuta.

**Cada tarea puede tener su propio rol.**




Permite que la plantilla cree recursos sin darle esos permisos a la persona.

**Principio: quien despliega no necesita ser administrador.**




Lo crea y define el propio servicio. No puedes cambiar sus permisos.

**Se reconoce por el prefijo AWSServiceRoleFor.**

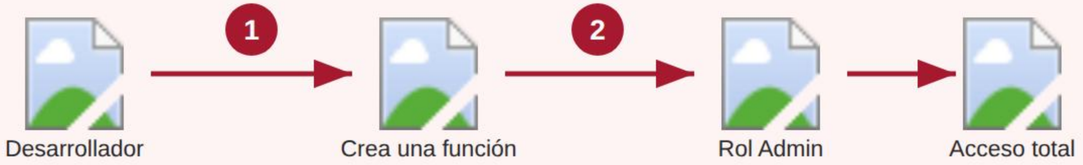


*Fuente: adaptado de AWS IAM User Guide — Service roles y service-linked roles*

# iam:PassRole: quién puede entregar qué rol

Entregar un rol a un servicio equivale a usar sus permisos: hay que acotarlo por rol y por servicio.

## Mal: PassRole sobre cualquier rol

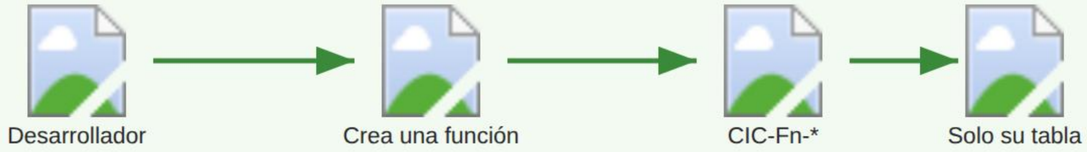



```
"Action": "iam:PassRole",
"Resource": "*"
```

El desarrollador no es administrador, pero entrega el rol Admin a una función suya. Su código corre con ese poder: escalada de privilegios.

**Riesgo: cualquiera puede llegar a ser administrador**

## Bien: PassRole acotado




```
"Action": "iam:PassRole",
"Resource": "arn:aws:iam::111111111111:role/CIC-Fn-*",
"Condition": {"StringEquals": {
  "iam:PassedToService": "lambda.amazonaws.com"}}
```

Solo puede entregar roles con ese prefijo y solo a Lambda. El rol Admin queda fuera de su alcance.

**Permiso explícito por rol y por servicio de destino**

**Regla: quien puede crear recursos y entregarles un rol hereda lo que el rol pueda hacer.**

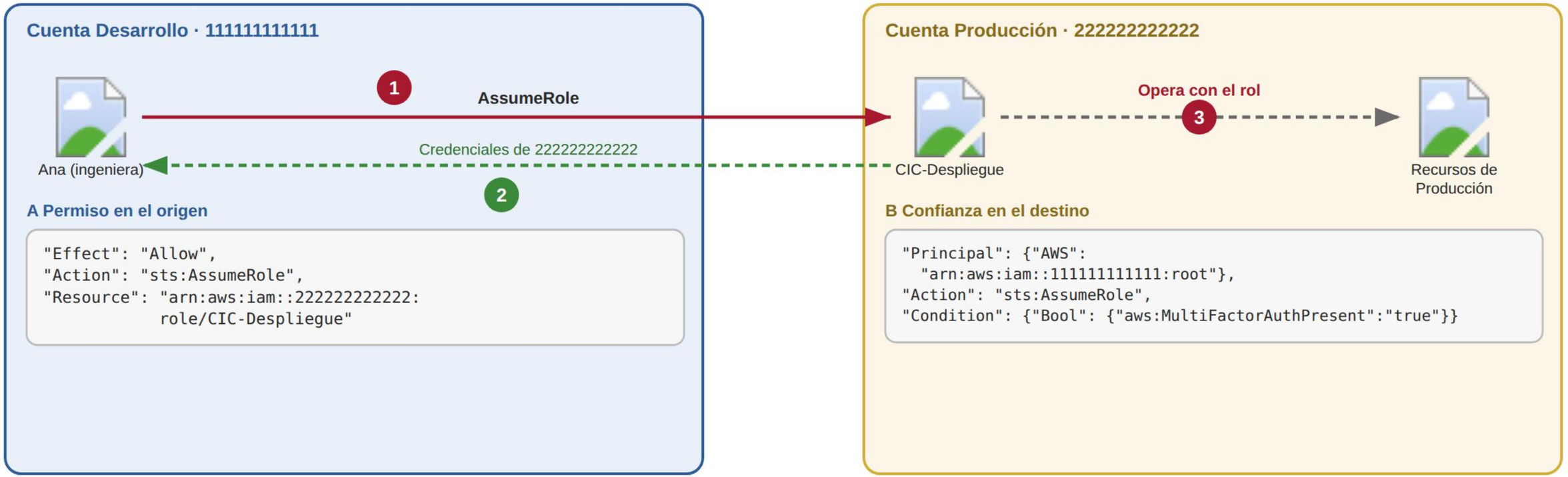
Por eso iam:PassRole se revisa tanto como un permiso de administrador.



*Fuente: adaptado de AWS IAM User Guide — Granting a user permissions to pass a role*

# Acceso entre cuentas: cada lado pone su parte

Producción confía en Desarrollo y Desarrollo permite pedir el rol: sin compartir ninguna llave.




**Dos llaves, dos dueños**

Entre cuentas deben coincidir A y B: el origen permite pedir el rol y el destino confía en el origen.

Si falta una, el resultado es acceso denegado. Producción decide quién entra aunque Desarrollo cambie.

**Sin ninguna llave compartida entre cuentas.**

**Con terceros: ExternalId**

Un proveedor externo asume tu rol con un identificador secreto: evita el problema del suplente confuso.



*Fuente: adaptado de AWS IAM User Guide — Cross-account access using roles*

# Quién fue: sesiones que dejan rastro en CloudTrail

Un rol compartido sigue siendo auditable si cada sesión lleva el nombre de la persona que lo asumió.




### Registro de CloudTrail (extracto)

```
"eventName": "DeleteObject",
"userIdentity.type": "AssumedRole",
"userIdentity.arn": "arn:aws:sts::222222222222:
    assumed-role/CIC-Despliegue/ana.perez",
"userIdentity.sourceIdentity": "ana.perez"
```

**Tres campos que responden ¿quién fue?**

type: AssumedRole distingue una sesión de un usuario IAM permanente.

arn: rol y nombre de sesión, juntos.

sourceIdentity: fijo; no cambia en la sesión y viaja al encadenar roles.

**Compartir un rol no es compartir anonimato**

Si exiges un nombre de sesión o SourceIdentity al asumir, cada acción en la cuenta queda atada a una persona real. Con un usuario compartido y una llave eterna esa trazabilidad se pierde por completo.



*Fuente: adaptado de AWS IAM User Guide — Monitor and control actions taken with assumed roles*

DECISIÓN

# ¿Dónde corre tu código? Así eliges la credencial

Casi siempre hay una alternativa a la llave de largo plazo: la pregunta es dónde vive quien llama.

**¿Dónde corre quien necesita acceso?**



**EC2, ECS o Lambda**

Rol: perfil de instancia, rol de tarea o de ejecución



**Laptop de una persona**

IAM Identity Center y aws sso login: sesión corta



**Pipeline de CI/CD**

Federación OIDC hacia un rol (sin secretos en el CI)



**Servidor en la planta (on-premises)**

IAM Roles Anywhere: certificado X.509 a sesión temporal



**Otra cuenta o un tercero**

Rol entre cuentas, con ExternalId para terceros



**Ninguna de las anteriores**

Última opción: llave rotada, mínima, vigilada



*Fuente: adaptado de AWS IAM Best Practices y documentación de IAM Roles Anywhere*

CASO APLICADO

# CIC elimina sus llaves de largo plazo

Catorce llaves repartidas por la empresa se convierten en una sola, vigilada.

**Llaves de acceso de largo plazo activas en la Corporación Industrial del Caribe (cifras ilustrativas)**




**IAM Identity Center**
reemplaza a las 6 llaves

**IAM Roles Anywhere**
reemplaza a las 4 llaves

**OIDC hacia un rol**
reemplaza a las 3 llaves

**Llave rotada cada 90 días**
reemplaza a las 1 llave

**Cómo lo hicieron**

1 Reporte de credenciales para inventariar llaves
2 Último uso por llave para saber qué sobra
3 Sustituir por rol, SSO u OIDC
4 Desactivar y, tras un período, eliminar

**Antes**
**14 llaves**

**Después**
**1 llave**

**Resultado: de 14 puntos de fuga a 1 vigilado, sin frenar el trabajo de ningún equipo.**



*Fuente: adaptado de AWS IAM Best Practices; cifras ilustrativas del caso CIC*

# Lo que queda de la Parte 1

Sin llaves eternas, cada identidad actúa con sesiones cortas y deja rastro.

1



## Los roles no tienen llaves

Se asumen: confianza para entrar, permisos para actuar.

2



## STS entrega sesiones

Cuatro piezas, 15 minutos a 12 horas, renovación automática.

3



## Cada carga tiene su rol

Perfil de instancia, rol de tarea, de ejecución o de servicio.

4



## Entre cuentas, dos firmas

Permiso en origen, confianza en destino, PassRole acotado.

## Siguiente: Parte 2 · Políticas a fondo

Ya sabes quién entra. Ahora decidimos qué puede hacer: políticas de identidad y de recurso, condiciones, límites de permisos, la lógica completa de evaluación y cómo llegar al mínimo privilegio paso a paso.





*Fuente: resumen de la Parte 1; AWS IAM User Guide*

ENCUADRE

# Anatomía de una política: cinco piezas

Casi todo lo que verás hoy se resuelve leyendo bien estas cinco piezas.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "LeerReportes",
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:ListBucket"],
    "Resource": ["arn:aws:s3:::cic-reportes",
                 "arn:aws:s3:::cic-reportes/*"],
    "Condition": {"IpAddress":
       {"aws:SourceIp": "203.0.113.0/24"}}
  }]
}
```

**Version** — Versión del lenguaje de políticas. Usa siempre 2012-10-17.

**Effect** — Allow o Deny. Lo que no se permite, queda negado por defecto.

**Action** — Qué operaciones: servicio y verbo, como s3:GetObject.

**Resource** — Sobre qué: el ARN exacto del recurso.

**Condition** — Cuándo aplica: IP, MFA, etiquetas, hora, región.



### Los comodines son deuda

Action y Resource con asterisco equivalen a ser administrador. Cada asterisco es una pregunta pendiente.



### El ARN es la dirección exacta

arn : partición : servicio : región : cuenta : recurso. Cuanto más exacto, menos riesgo.



### Sin Allow, todo se niega

Un Deny explícito vence a cualquier Allow. Principal solo aparece en políticas de recurso.

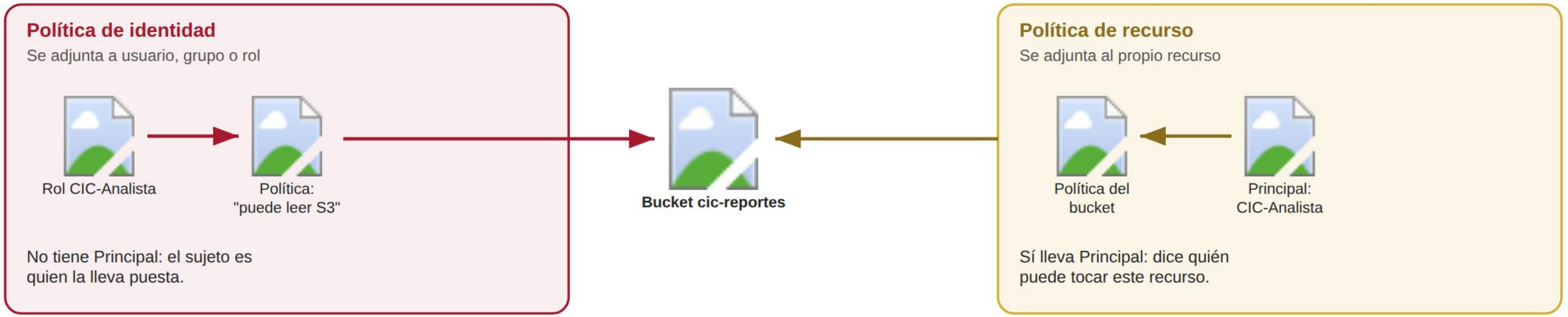


*Fuente: adaptado de AWS IAM User Guide — Policies and permissions in IAM*

DOS CAMINOS

# Políticas de identidad frente a políticas de recurso

Una vive en quien pide, la otra en lo que se pide; entre cuentas hacen falta ambas.




### Misma cuenta: basta con una

Si la identidad permite O la política de recurso permite al principal, y nadie niega, se autoriza.

### Entre cuentas: deben coincidir las dos

La identidad (cuenta origen) Y el recurso (cuenta destino) deben permitir. Con una sola, hay acceso denegado.

**Se llaman políticas de recurso, y también se usan en:**





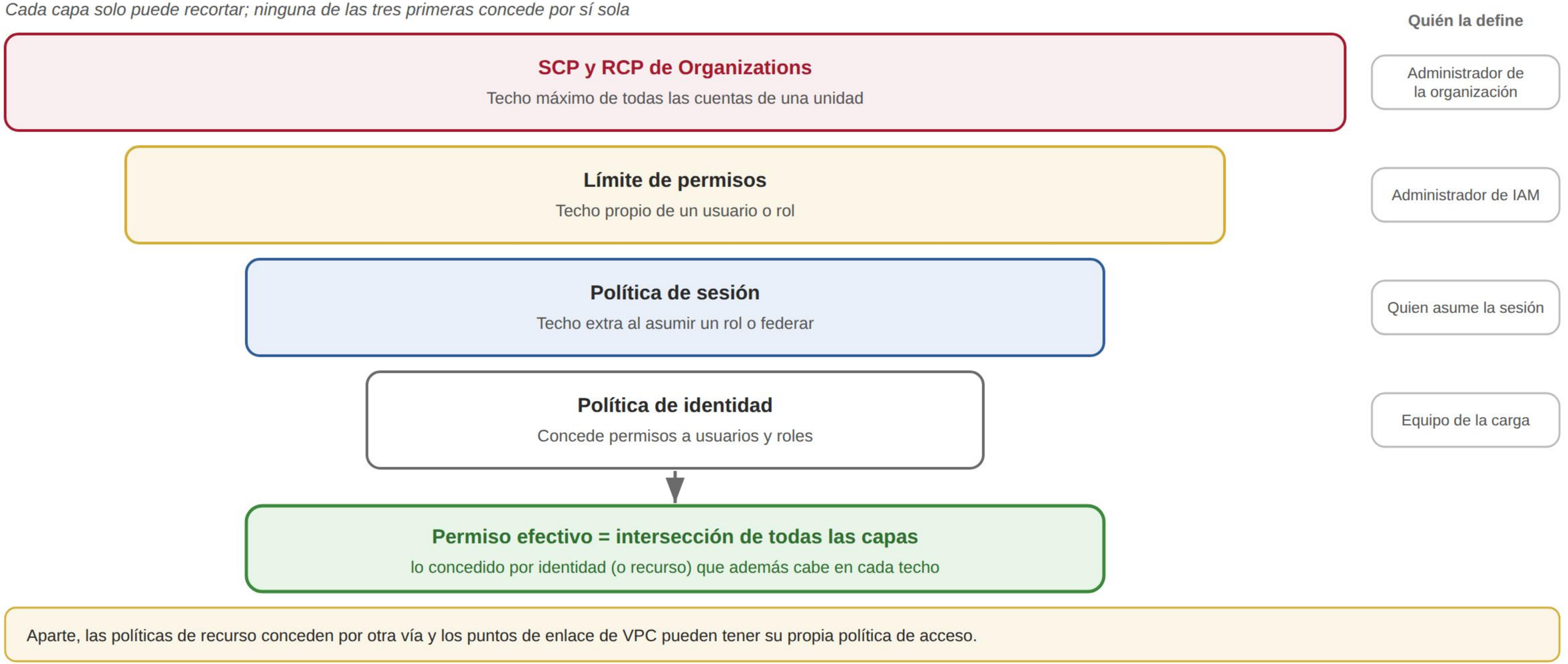

*Fuente: adaptado de AWS IAM User Guide — Identity-based and resource-based policies*

MAPA DE POLÍTICAS

# Las capas que se suman para decidir un permiso

Unas conceden y otras ponen techo: el permiso real es lo que cabe en todas a la vez.

*Cada capa solo puede recortar; ninguna de las tres primeras concede por sí sola*




Aparte, las políticas de recurso conceden por otra vía y los puntos de enlace de VPC pueden tener su propia política de acceso.



*Fuente: adaptado de AWS IAM User Guide — Policy evaluation logic y tipos de políticas*

# Condiciones: IP, MFA, etiquetas y hora

Una condición convierte un permiso general en uno que solo vale en el contexto correcto.



### Desde dónde: IP

Solo desde la red de la oficina o la VPN de la planta.

```
"IpAddress": {
  "aws:SourceIp": "203.0.113.0/24"}
```



### Con qué seguridad: MFA

Operaciones sensibles solo con segundo factor.

```
"Bool": {
  "aws:MultiFactorAuthPresent": "true"}
```



### Sobre qué: etiquetas

Solo recursos con la etiqueta entorno igual a dev.

```
"StringEquals": {
  "aws:ResourceTag/entorno": "dev"}
```



### Cuándo: hora

Acceso temporal para una ventana de trabajo.

```
"DateLessThan": {
  "aws:CurrentTime": "2026-10-31T23:59:59Z"}
```

**Cómo se lee: operador + clave + valor**

Todas las condiciones de un bloque deben cumplirse (Y). Varios valores de una misma clave se combinan con O.

Otras claves muy útiles: aws:RequestedRegion, aws:SecureTransport, aws:SourceVpce, aws:PrincipalOrgID.



*Fuente: adaptado de AWS IAM User Guide — IAM JSON policy elements: Condition*

SEMANA 9 · IDENTIDAD AVANZADA

# Mínimo privilegio a fondo y federación

Parte 2 de 3 · Unidad 2

**Ing. Rodolfo Cañas Cervantes**

Universidad de la Costa (CUC) · Ingeniería de Sistemas

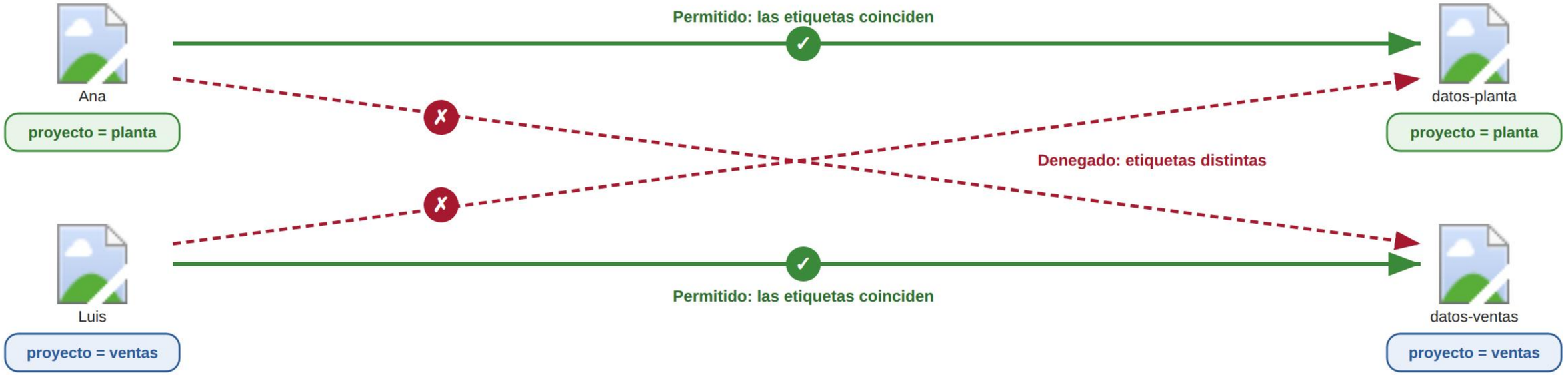
Semana 9 · Periodo 2026-2 · Barranquilla, Colombia

Políticas con condiciones, cinco puertas de evaluación, Access Analyzer y federación con SAML y OpenID Connect

# ABAC: permisos que siguen a las etiquetas

Una política con etiquetas escala mejor que cien políticas por proyecto.

## Una sola política para todos los proyectos




```
"Action": ["s3:GetObject", "s3:PutObject"],
"Resource": "*",
"Condition": {"StringEquals": {
  "aws:ResourceTag/proyecto":
  "${aws:PrincipalTag/proyecto}"}}
```

### ABAC: control de acceso por atributos

Un proyecto nuevo no exige política nueva: basta etiquetar a la persona y a los recursos.
El permiso crece con la empresa, no con el número de políticas. Requiere gobernar las etiquetas (parte 4) para que nadie se las cambie.



*Fuente: adaptado de AWS IAM User Guide — ABAC for AWS y IAM tutorial: tags*

GUARDARRAÍLES

# Tres Deny con condición que protegen a todos

Un Deny condicionado no concede nada: solo cierra puertas que ninguna otra política puede abrir.



### Sin MFA no se borra

Deny de borrados si la sesión no tiene segundo factor

**DENY** gana a cualquier Allow

```
"Effect": "Deny",
"Action": ["ec2:TerminateInstances","rds:DeleteDBInstance"],
"Condition": {"BoolIfExists":
  {"aws:MultiFactorAuthPresent": "false"}}
```



### Solo regiones aprobadas

Deny de todo lo que se solicite fuera de São Paulo y Norte de Virginia

**DENY** gana a cualquier Allow

```
"Effect": "Deny", "NotAction": ["iam:*","sts:*"],
"Resource": "*",
"Condition": {"StringNotEquals": {"aws:RequestedRegion":
  ["sa-east-1","us-east-1"]}}
```



### Siempre cifrado en tránsito

Deny de cualquier petición a S3 que no use TLS

**DENY** gana a cualquier Allow

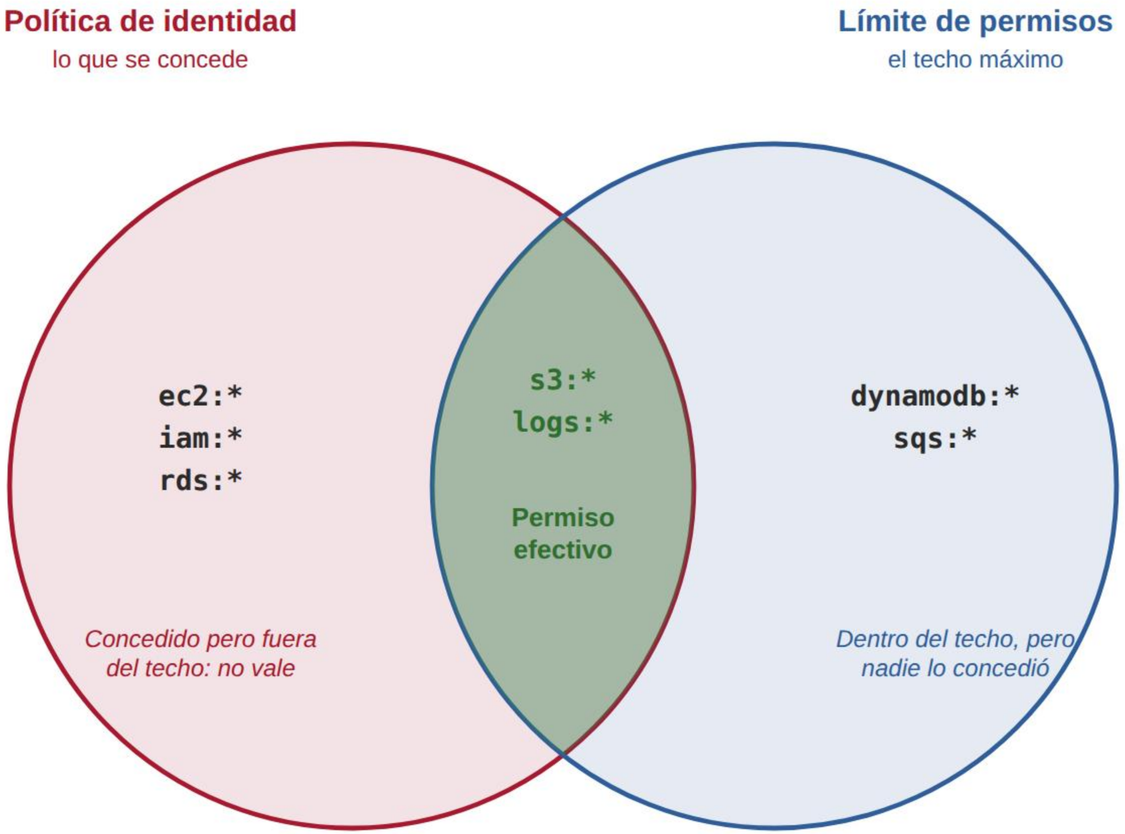
```
"Effect": "Deny", "Action": "s3:*",
"Resource": ["arn:aws:s3:::cic-*","arn:aws:s3:::cic-*/*"],
"Condition": {"Bool":
  {"aws:SecureTransport": "false"}}
```



*Fuente: adaptado de AWS IAM User Guide — Example policies y AWS Security Reference Architecture*

# Límites de permisos: el techo de una identidad

El permiso efectivo es la intersección entre lo concedido y lo que el límite permite.




**El límite no concede nada**

Solo marca hasta dónde puede llegar una identidad.

**Se aplica a usuarios y roles**

No existe límite de permisos para grupos.

**Recorta sin editar la política**

Si alguien se agrega ec2:*, el límite sigue mandando.

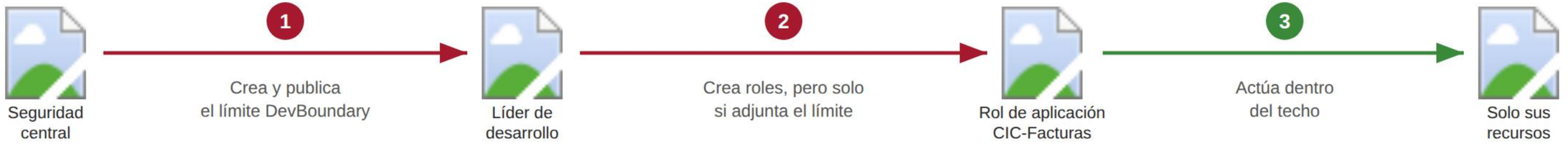


*Fuente: adaptado de AWS IAM User Guide — Permissions boundaries for IAM entities*

DELEGACIÓN

# Delegar la creación de roles sin perder el control

Seguridad fija el techo una vez; los equipos crean roles propios sin poder superarlo.




```
// política del líder: puede crear roles solo con límite
"Action": ["iam:CreateRole","iam:AttachRolePolicy"],
"Condition": {"StringEquals": {
  "iam:PermissionsBoundary":
  "arn:aws:iam::111111111111:policy/DevBoundary"}}
```

**Intentos que fallan**

Crear un rol sin límite

Adjuntar AdministratorAccess: sigue acotado por el límite

Quitar el límite o editarlo (protegido por un Deny)

**DENEGADO**

## Delegar sin perder el control

1. Seguridad define el techo una sola vez.
2. Los equipos crean sus propios roles, sin esperar un ticket.
3. Todo rol nuevo nace acotado; no hay escalada posible.
4. Seguridad audita un límite, no mil roles.

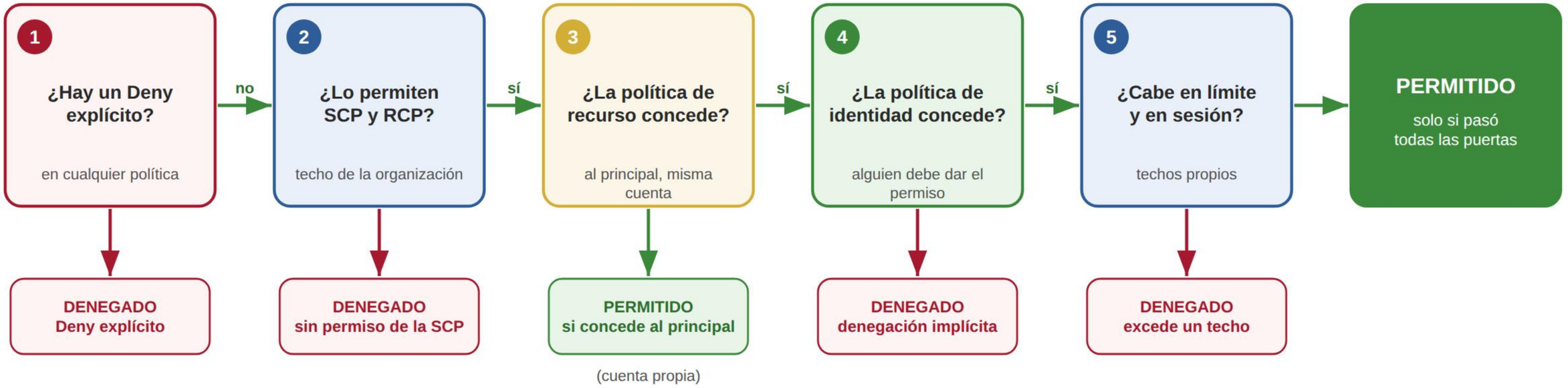


*Fuente: adaptado de AWS IAM User Guide — Delegating responsibility using permissions boundaries*

EVALUACIÓN

# Lógica de evaluación: cinco puertas en orden

La solicitud se aprueba solo si pasa todas las puertas; el Deny explícito corta el camino de inmediato.




## Cómo recordarlo

1. **Por defecto, todo se niega.**
   Sin Allow no hay acceso.
2. **Un Deny explícito siempre gana.**
   En cualquier política, de cualquier tipo.
3. **Los techos deben permitir todos.**
   SCP, RCP, límite y sesión.
4. **Alguien debe conceder.**
   Identidad o recurso, según el caso.



*Fuente: adaptado de AWS IAM User Guide — Policy evaluation logic (diagrama de flujo)*

PRÁCTICA

# Cuatro solicitudes: ¿quién decide en cada caso?

Aplicar las cinco puertas a casos concretos es la mejor manera de dejar de adivinar.

**Solicitud: s3:GetObject sobre un bucket de la empresa**

| | | Política de identidad | Política del bucket | SCP de la organización | Resultado |
|---|---|---|---|---|---|
| A |  **Ana, misma cuenta** | Allow | Deny | Allow | → **DENEGADO** el Deny explícito del bucket gana |
| B |  **Rol de la app, misma cuenta** | Sin sentencia | Allow al rol | Allow | → **PERMITIDO** el recurso concede dentro de la cuenta |
| C |  **Luis, misma cuenta** | Allow | Sin sentencia | No lo permite | → **DENEGADO** la SCP no permite el servicio |
| D |  **Analista de otra cuenta** | Allow | Sin sentencia | Allow | → **DENEGADO** entre cuentas faltó el permiso del recurso |



*Fuente: adaptado de AWS IAM User Guide — Policy evaluation logic: ejemplos de casos*

DETECCIÓN

# IAM Access Analyzer: lo que está expuesto

El analizador responde la pregunta que nadie se hace: quién de afuera puede llegar a mis recursos.




Access Analyzer

*analiza*

**Hallazgos (ejemplos)**

| | |
|---|---|
| Alto | Bucket cic-publico permite acceso a cualquier persona en Internet |
| Alto | Rol CIC-Integra confía en la cuenta externa 999999999999 |
| Medio | Usuario jperez sin actividad en los últimos 90 días |
| Medio | Clave KMS compartida con otra cuenta fuera de la organización |
| Bajo | Política con permisos de servicios nunca usados |

**Acceso externo**
quién de afuera llega a tus recursos

**Acceso no utilizado**
roles, llaves y permisos sin uso

**Validar y comprobar**
sintaxis, advertencias y controles propios

**Generar políticas**
desde la actividad de CloudTrail

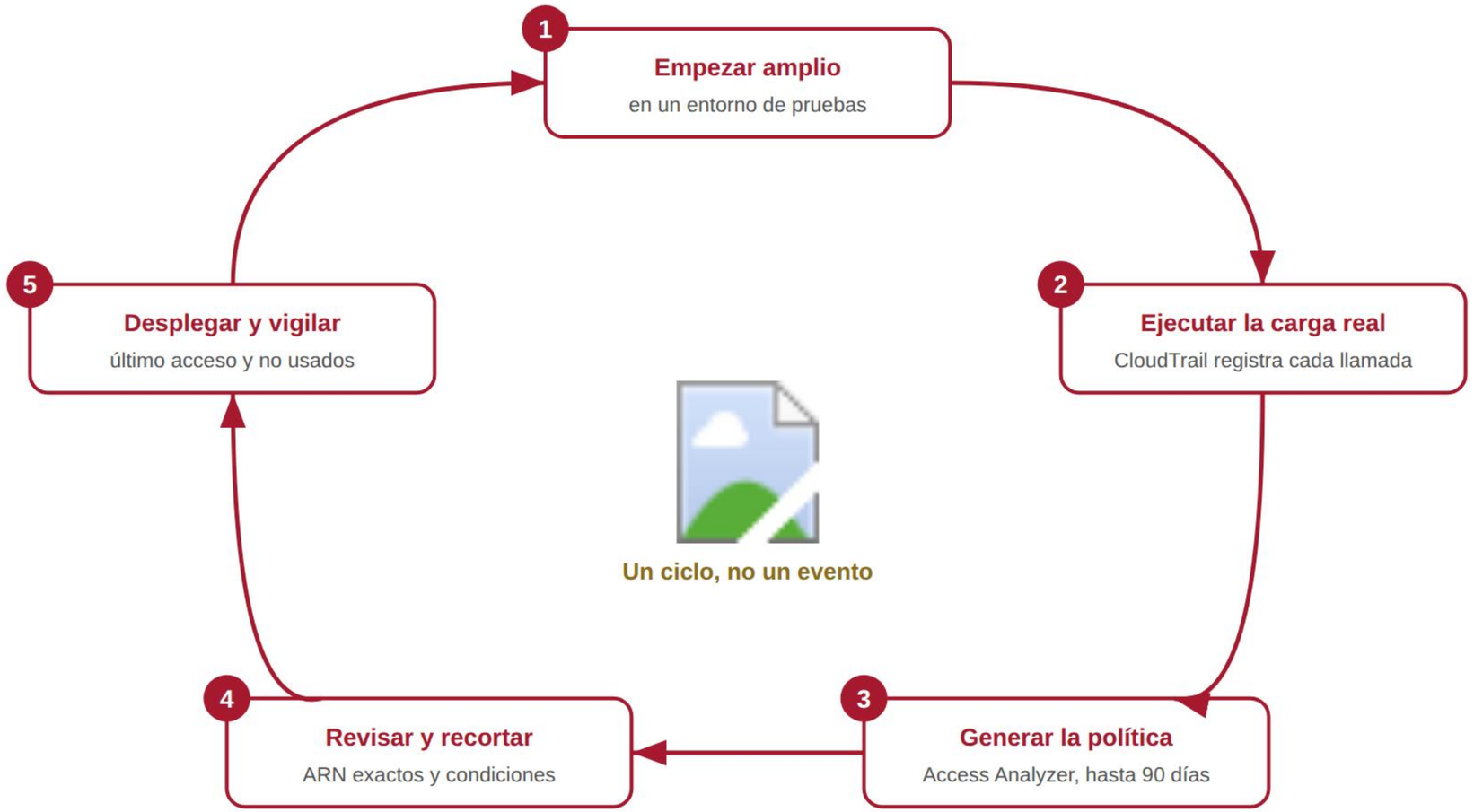


*Fuente: adaptado de AWS IAM Access Analyzer — documentación y casos de uso*

MÉTODO

# Mínimo privilegio como ciclo de mejora continua

Se empieza con permisos amplios en pruebas y se recortan con datos reales, no con suposiciones.




## Qué aporta cada herramienta

**CloudTrail**

el registro de lo que realmente se llamó.

**Último acceso**

qué servicios y acciones se usaron por última vez.

**Generar política**

borrador con solo las acciones observadas, para revisar.

**Acceso no utilizado**

hallazgos de roles, llaves y permisos sin uso.

**Atención: el borrador no es la versión final. Hay que revisarlo y poner ARN exactos.**



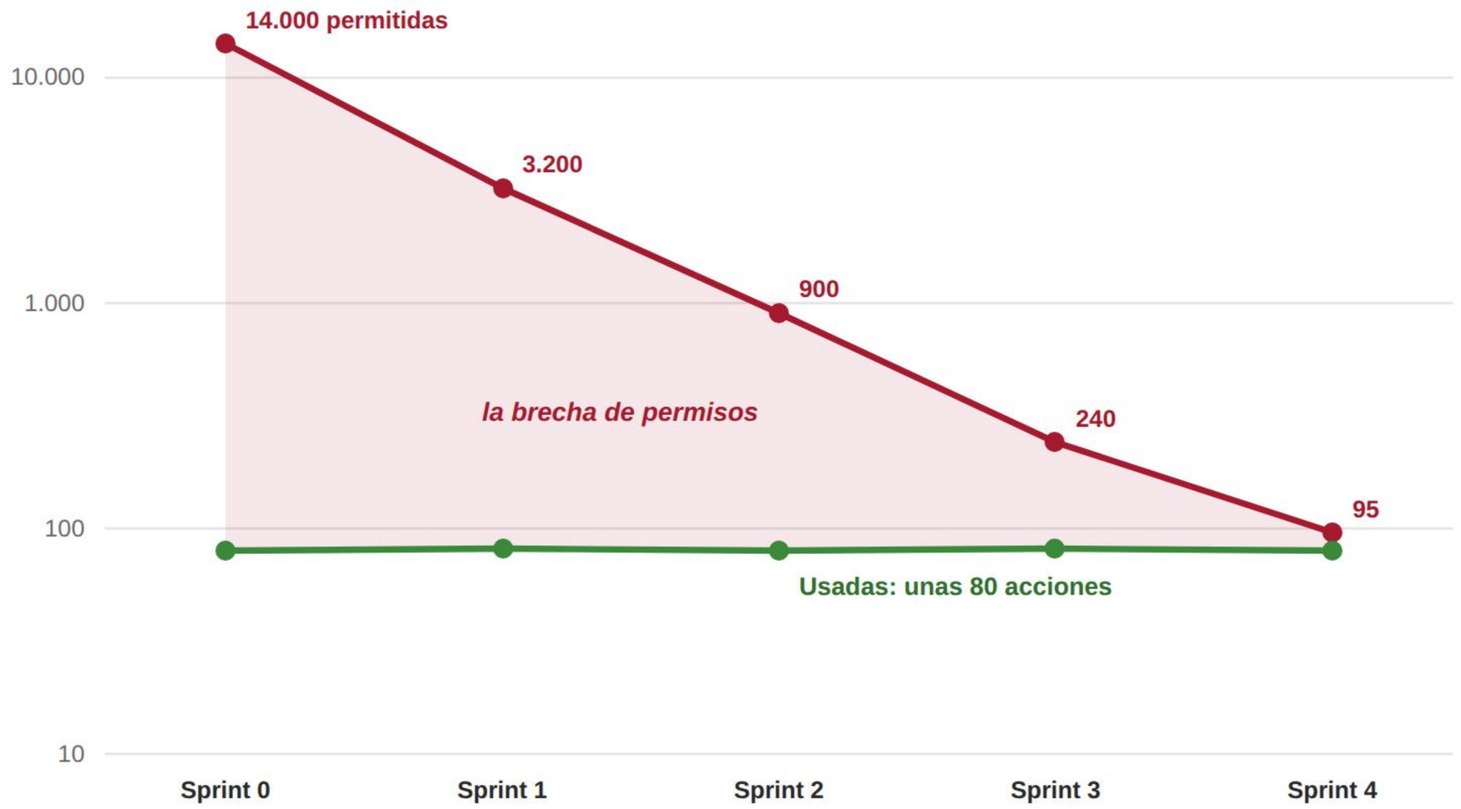
*Fuente: adaptado de AWS IAM User Guide — Generate policies based on CloudTrail events*

CASO APLICADO

# CIC reduce la brecha de permisos en cuatro sprints

La distancia entre lo concedido y lo usado se cierra con datos, no con intuición.

**Acciones permitidas frente a acciones realmente usadas (escala logarítmica, cifras ilustrativas)**




## CIC: de administrador a justo

**0 Sprint 0**
AdministratorAccess en todo

**1 Sprint 1**
Políticas administradas por servicio

**2 Sprint 2**
Política generada desde CloudTrail

**3 Sprint 3**
Condiciones y límite de permisos

**4 Sprint 4**
Revisión continua de lo no usado

*Cifras ilustrativas*



*Fuente: adaptado de AWS Well-Architected, pilar Seguridad (SEC03); cifras ilustrativas*

# Lo que queda de la Parte 2

Permisos justos se construyen con condiciones, techos y mejora continua.

1



## Lee la política por dentro

Effect, Action, Resource y Condition; sin comodines innecesarios.

2



## Usa condiciones

IP, MFA, etiquetas y hora; Deny con condición como guardarraíl.

3



## Pon techos

Límites de permisos y SCP; el permiso real es la intersección.

4



## Itera con datos

Access Analyzer, CloudTrail y último acceso para cerrar la brecha.

## Siguiente: Parte 3 · Personas y aplicaciones

Hasta ahora las identidades vivían dentro de AWS. En la Parte 3 conectamos al directorio de la empresa con federación y SSO, damos identidad a los clientes de una aplicación con Cognito y protegemos secretos y claves de cifrado.





*Fuente: resumen de la Parte 2; AWS IAM User Guide*

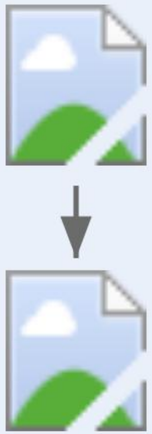
ENCUADRE

# Tres tipos de identidad, tres herramientas

Personas, cargas de trabajo y clientes de una app se resuelven con servicios distintos.

## Personas

Empleados y contratistas

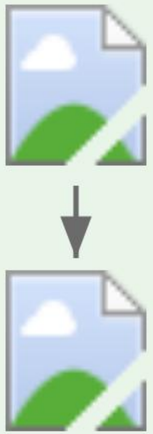


**IAM Identity Center**
**+ proveedor de identidad**

Sesión corta en la consola
y en la línea de comandos

## Cargas de trabajo

Aplicaciones y servidores

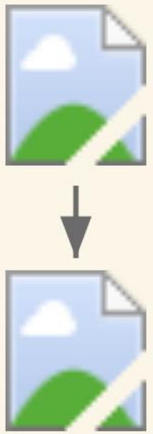


**Roles de IAM**
**(Parte 1)**

Credenciales temporales
renovadas por el servicio

## Clientes de tu aplicación

Usuarios de un producto



**Amazon Cognito**

Tokens JWT y, si hace falta,
credenciales temporales

**Regla de diseño: cada tipo de identidad vive en su propio directorio.**

Los empleados no se mezclan con los clientes: ciclo de vida, controles de seguridad y volumen son distintos.

Ejemplos de CIC: ingenieros, la app de pedidos y los distribuidores.



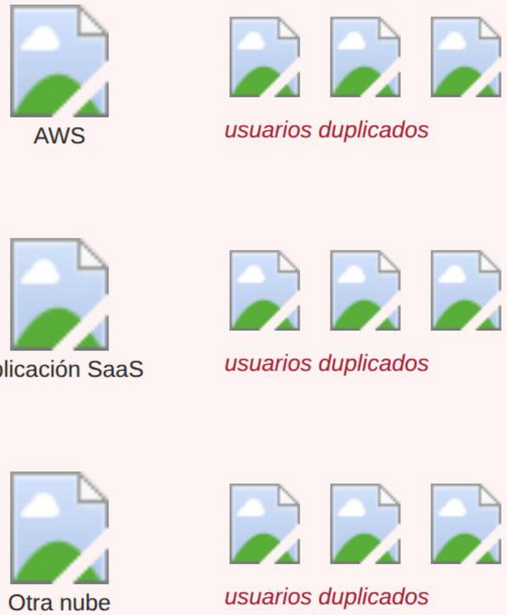
*Fuente: adaptado de AWS Well-Architected, pilar Seguridad (SEC02): identity management*

# Identidad federada: un solo directorio, muchas puertas

AWS y las demás plataformas confían en el proveedor de identidad de la empresa.

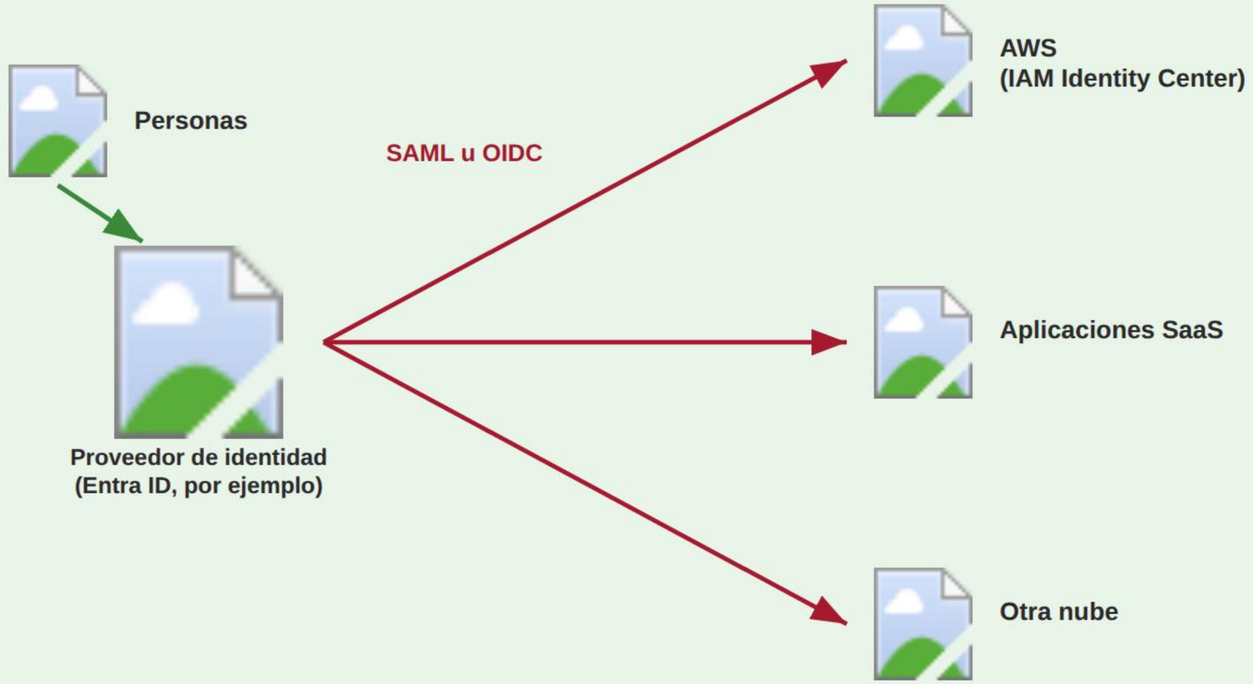
## Sin federación

Un usuario y una contraseña por plataforma




**Altas, bajas y MFA, repetidas tres veces**

## Con federación



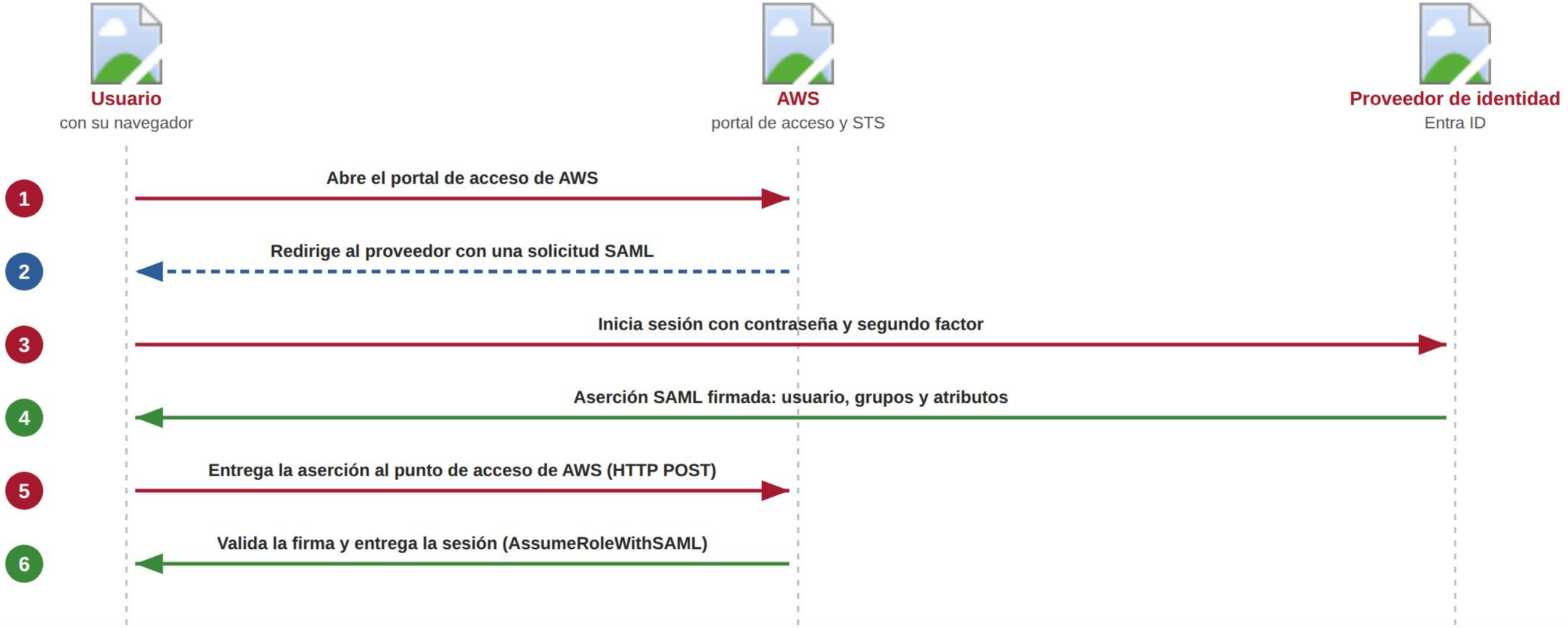

**Altas y bajas en un solo lugar · MFA y acceso condicional centralizados · AWS confía en el proveedor y nunca guarda la contraseña**



*Fuente: adaptado de AWS IAM User Guide — Identity providers and federation*

# SAML: el inicio de sesión federado paso a paso

El navegador lleva la aserción firmada del proveedor hasta AWS; la contraseña nunca viaja a AWS.




La confianza se configura una vez, intercambiando metadatos (certificado y direcciones) entre el proveedor y AWS.

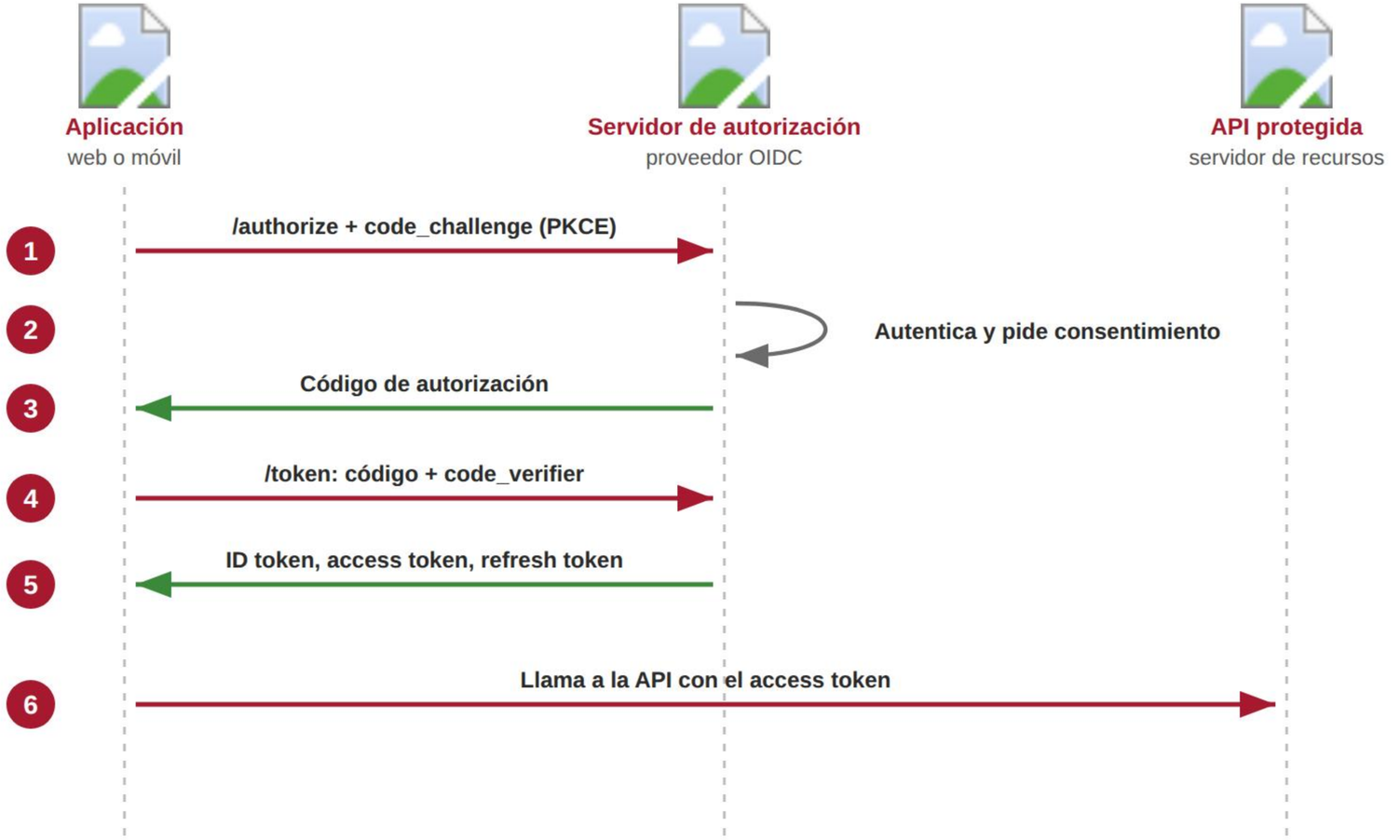
Después, AWS solo comprueba la firma: no conoce la contraseña y no necesita llamar al directorio en cada inicio de sesión.



*Fuente: adaptado de AWS IAM User Guide — About SAML 2.0-based federation*

# OIDC y OAuth 2.0: tokens para aplicaciones modernas

La aplicación recibe tokens firmados que dicen quién es el usuario y qué puede pedir.




## Un JWT son tres partes

**Encabezado**
algoritmo y tipo

**Contenido**
sub, iss, aud, exp, grupos

**Firma**
prueba de que no se alteró

### Qué tokens recibe la app

ID token: quién es la persona.
Access token: qué puede pedir a la API.
Refresh token: renueva sin pedir clave.

**Con PKCE no hace falta secreto en la app.**

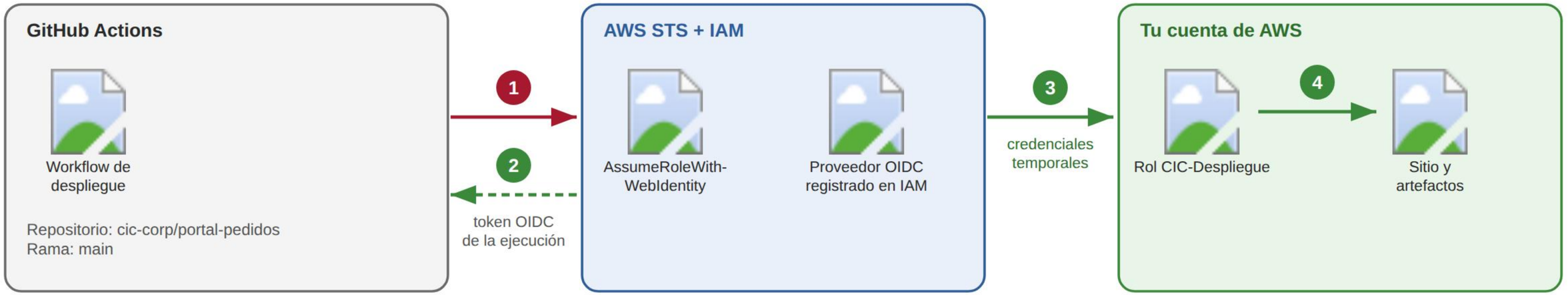
**OAuth 2.0 autoriza (qué puedes hacer); OpenID Connect añade la identidad (quién eres) encima de OAuth 2.0.**



*Fuente: adaptado de OpenID Connect Core 1.0 y RFC 7636 (PKCE); AWS IAM User Guide — OIDC federation*

# Pipelines sin llaves: GitHub Actions hacia AWS

El pipeline presenta un token firmado de GitHub y recibe una sesión de AWS que caduca.




```
"Principal": {"Federated":
  "arn:aws:iam::111111111111:oidc-provider/token.actions.githubusercontent.com"},
"Action": "sts:AssumeRoleWithWebIdentity",
"Condition": {"StringEquals": {
  "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
  "token.actions.githubusercontent.com:sub":
  "repo:cic-corp/portal-pedidos:ref:refs/heads/main"}}
```

### La condición sub es la clave

Solo ese repositorio y esa rama pueden asumir el rol. Sin esta condición, cualquier repositorio de GitHub podría intentarlo.

**Resultado: ningún secreto guardado en el pipeline y sesiones de corta duración.**

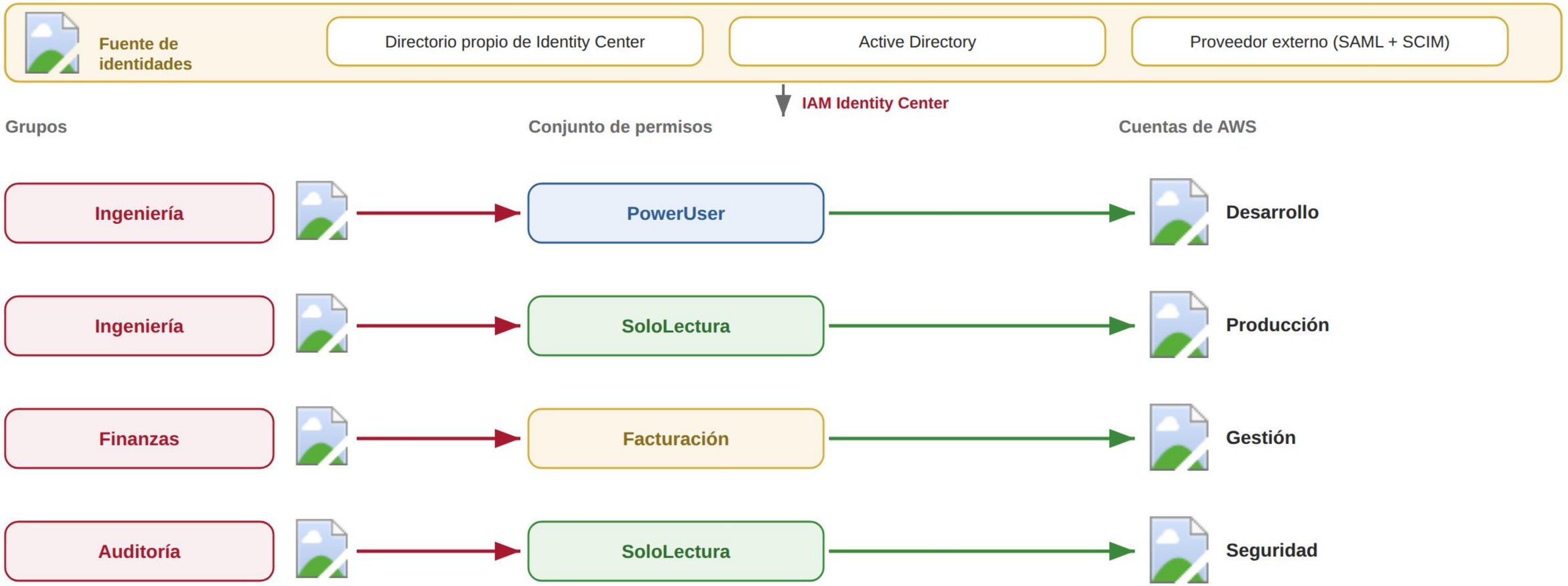
**El mismo patrón sirve para GitLab, Kubernetes y cualquier proveedor compatible con OIDC.**



*Fuente: adaptado de GitHub Docs — Configuring OpenID Connect in AWS y AWS IAM User Guide*

# IAM Identity Center: grupos, permisos y cuentas

Una persona entra una vez y ve solo las cuentas y roles que su grupo tiene asignados.



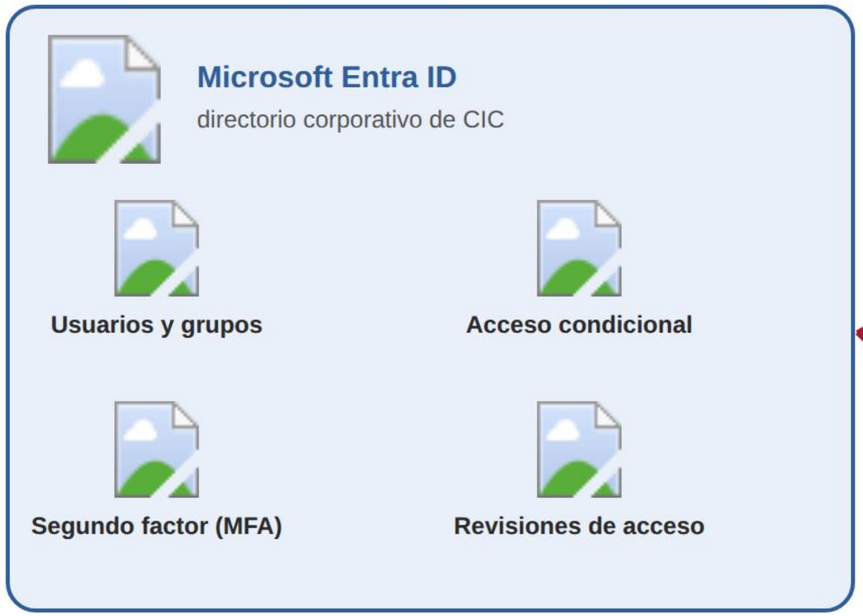

*Cada asignación = grupo + conjunto de permisos + cuenta. El servicio crea en la cuenta un rol llamado AWSReservedSSO_<conjunto>_<código>.*



*Fuente: adaptado de AWS IAM Identity Center User Guide — Multi-account permissions*

# Entra ID como fuente: un ejemplo de proveedor

La persona se da de alta una vez y su acceso llega a AWS, a Azure y a las aplicaciones SaaS.







## Ciclo de vida de una persona

**Ingreso**
Se crea en Entra ID y entra a un grupo

→

**Cambio de área**
Cambia de grupo: cambian sus accesos

→

**Salida**
Se deshabilita una vez: se cierra todo

**Entra ID es un ejemplo: el mismo diseño funciona con Okta, Google Workspace o Active Directory.**



*Fuente: adaptado de Microsoft Learn — Entra ID e IAM Identity Center (provisión con SCIM)*

# Cognito: user pools frente a identity pools

Uno responde quién eres; el otro, qué recursos de AWS puedes usar directamente.



**Amazon Cognito**



Usuario de la aplicación

## User pool

Directorio de usuarios

**¿Quién eres? (autenticación)**

Registro, inicio de sesión y recuperación

MFA y protección contra credenciales robadas

Acceso con Google, Apple o SAML/OIDC

Interfaz de inicio de sesión alojada

**Entrega: tokens JWT**

## Identity pool

Identidades federadas

**¿Qué puedes tocar en AWS? (autorización)**

Recibe un token de confianza (user pool, Google, SAML, OIDC)

Lo cambia por credenciales temporales de AWS

Usa un rol para usuarios autenticados

y otro, más limitado, para invitados

**Entrega: credenciales de STS**



rol con permisos mínimos

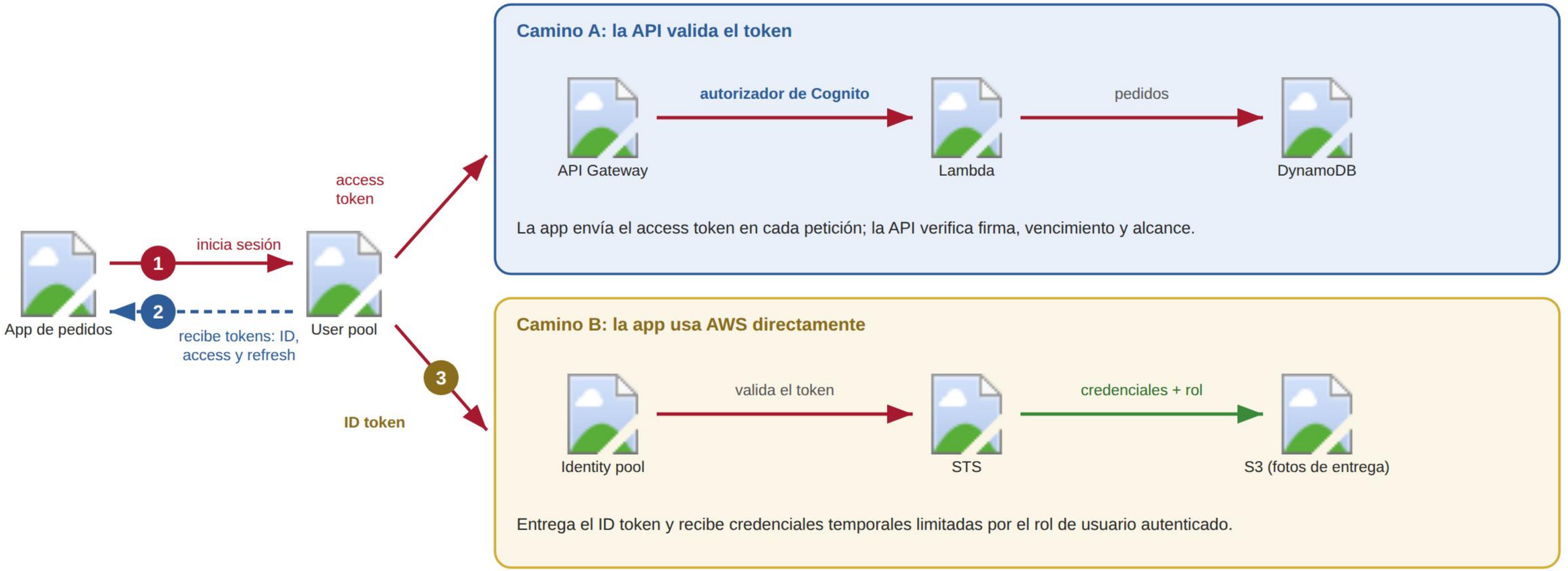
**Se pueden usar por separado o juntos. Una API suele necesitar solo el user pool; acceso directo a S3 desde la app añade el identity pool.**



*Fuente: adaptado de Amazon Cognito Developer Guide — User pools and identity pools*

# Dos caminos para una app: API o acceso directo

Los tokens del user pool abren la API; el identity pool los cambia por credenciales de AWS.




**Con identity pool conviene que el rol limite la ruta del objeto por usuario, usando la variable de identidad de Cognito en la política.**
Así cada distribuidor solo ve sus propias carpetas.


*Fuente: adaptado de Amazon Cognito Developer Guide — Common Amazon Cognito scenarios*

SEMANA 9 · IDENTIDAD AVANZADA

# Secretos y gobernanza multi-cuenta

Parte 3 de 3 · Unidad 2

**Ing. Rodolfo Cañas Cervantes**

Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semana 9 · Periodo 2026-2 · Barranquilla, Colombia

Secretos y políticas de claves, y la gobernanza multi-cuenta con Organizations, SCP y costos visibles

# Secretos: del código a Secrets Manager

La aplicación pide la contraseña cuando la necesita, con su rol, y el servicio la rota sola.

## Mal: el secreto en el código

```
db_pass = "Cic#2024!"
# ¿y si el repo se filtra?
# ¿y quién la rota?
```


Repositorio


Laptop


Variable de entorno

**Se copia, se comparte y nunca se rota.**

## Bien: la app pide el secreto en el momento


App con rol

1 GetSecretValue


Secrets Manager

2 cifrado en reposo


Clave KMS



### Rotación automática cada 30 días

Una función Lambda crea una contraseña nueva, la cambia en la base de datos y marca la versión como actual.
La app no se entera.

*Versiones: AWSCURRENT, AWSPREVIOUS y AWSPENDING durante el cambio.*


Base de datos

## Quién puede leer el secreto

El rol de la app, con permiso secretsmanager:GetSecretValue sobre ese secreto y nada más.
Se puede añadir una política de recurso en el secreto.
La lectura queda en CloudTrail.

## Y si no necesita rotación

Parameter Store con SecureString guarda configuración y valores sensibles sencillos a menor costo, cifrados con KMS.
Elige Secrets Manager si quieres rotación.



*Fuente: adaptado de AWS Secrets Manager User Guide — Rotate secrets y Best practices*

# Políticas de claves: administrar no es usar

Quien gestiona la clave no puede descifrar datos, y quien descifra no puede cambiarla.



**Clave KMS**
**cic-datos**

```
// política de la clave (key policy)
{"Sid":"Habilitar políticas de IAM",
  "Principal":{"AWS":"arn:aws:iam::111111111111:root"},
  "Action":"kms:*","Resource":"*"}
{"Sid":"Administradores","Principal":{"AWS":".../CIC-SeguridadKms"},
  "Action":["kms:Create*","kms:Describe*","kms:Enable*",
            "kms:ScheduleKeyDeletion"],"Resource":"*"}
```

### La política de la clave manda

En KMS, el acceso depende primero de la política de la clave. La primera sentencia permite que además las políticas de IAM concedan permisos.



### Administradores de la clave

Gestionan, pero no usan

Pueden: crear, describir, habilitar, rotar, programar la eliminación.

**No pueden: Encrypt ni Decrypt**



### Usuarios de la clave

Usan, pero no administran

Pueden: Encrypt, Decrypt, ReEncrypt, GenerateDataKey.

**No pueden: borrar ni cambiar la clave**

### Condiciones que acotan el uso

kms:ViaService: solo a través de S3 o EBS, no directo. kms:EncryptionContext: el dato debe llevar la etiqueta esperada.

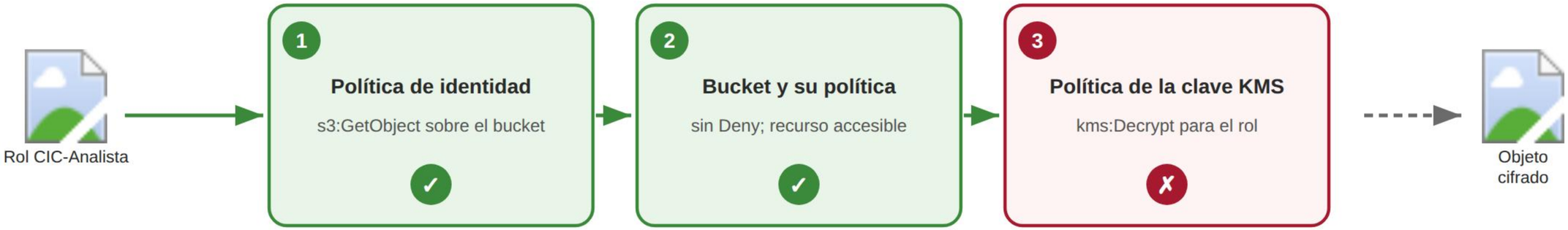


*Fuente: adaptado de AWS KMS Developer Guide — Key policies in AWS KMS*

PUNTO DE ENCUENTRO

# Un dato cifrado exige tres permisos, no uno

Si el acceso falla, revisa las tres puertas: identidad, recurso y clave.




**Resultado: AccessDenied aunque las puertas 1 y 2 estén bien**

```
User is not authorized to perform: kms:Decrypt on the resource (mensaje típico de AWS)
```

S3 llama a KMS en nombre del rol: el permiso de la clave debe existir también.

## Cómo depurar con método

### CloudTrail

Busca el evento fallido y su código de error

### Simulador de políticas

Prueba una acción sin ejecutarla

### Access Analyzer

Revisa políticas y accesos inesperados



*Fuente: adaptado de AWS KMS y Amazon S3 User Guide — Using SSE-KMS; IAM Policy Simulator*

CASO APLICADO

# CIC: quién entra y por cuál puerta

Cada tipo de persona o carga entra por su propia puerta y llega solo a las cuentas que le corresponden.

### Quién entra



**Empleados**
Entra ID + MFA

→ **IAM Identity Center**
conjuntos de permisos por grupo



**Contratistas**
invitados, con vencimiento

→ **IAM Identity Center**
acceso temporal y de solo lectura



**Distribuidores**
portal de pedidos

→ **Cognito user pool**
tokens JWT por distribuidor



**Planta Barranquilla**
servidores locales

→ **IAM Roles Anywhere**
certificado X.509 a sesión

*Cada flecha lleva a una sola puerta: no hay accesos paralelos escondidos.*

### Cuentas de AWS de CIC



**Desarrollo**
roles de ingeniería, sin datos reales



**Producción**
pedidos, facturación, planta



**Seguridad y registros**
CloudTrail y Access Analyzer centralizados



**Datos compartidos**
secretos y claves KMS gestionados

### Controles transversales

- Sin llaves de largo plazo (Parte 1)
- Mínimo privilegio con límites (Parte 2)
- Secretos rotados en Secrets Manager
- Claves KMS con funciones separadas



*Fuente: adaptado de AWS Well-Architected, pilar Seguridad; caso transversal CIC*

# Lo que queda de la Parte 3

Personas federadas, clientes en Cognito y secretos y claves bien gobernados.

1



## Federa a las personas

Un directorio, SAML u OIDC e IAM Identity Center con grupos y conjuntos de permisos.

2



## Pipelines sin secretos

OIDC con condiciones sobre repositorio y rama, y sesiones cortas.

3



## Separa a los clientes

User pool para identidad; identity pool para credenciales de AWS.

4



## Protege secretos y claves

Secrets Manager con rotación y políticas de clave con funciones separadas.

## Siguiente: Parte 4 · Gobernanza multi-cuenta y costos

Ya sabemos quién entra y qué puede hacer. Falta el techo de toda la organización: cuentas, SCP y guardarraíles, y la otra gran pregunta de la nube: cuánto cuesta. Etiquetas, presupuestos, calculadoras y TCO.





*Fuente: resumen de la Parte 3; AWS IAM Identity Center, Cognito y KMS*

ENCUADRE

# Por qué una empresa usa muchas cuentas de AWS

La cuenta es la frontera más fuerte de AWS: aísla permisos, costos y errores.

## Una sola cuenta para todo


Producción


Pruebas


Datos reales


Registros


Sandbox


Todos

**Un error de un desarrollador toca producción**

**Un solo radio de impacto**
**Una sola factura mezclada**
**Límites de servicio compartidos**

## Varias cuentas con propósito



**Producción**
pedidos y planta

**Aislada del error de otras cuentas**



**Desarrollo**
pruebas del equipo

**Un error aquí se queda aquí**



**Seguridad**
registros y auditoría

**Aislada del error de otras cuentas**



**Datos**
secretos y claves

**Aislada del error de otras cuentas**

### Aislamiento
el daño no se propaga

### Facturación
una cuenta, un centro de costo

### Límites
cuotas por cuenta

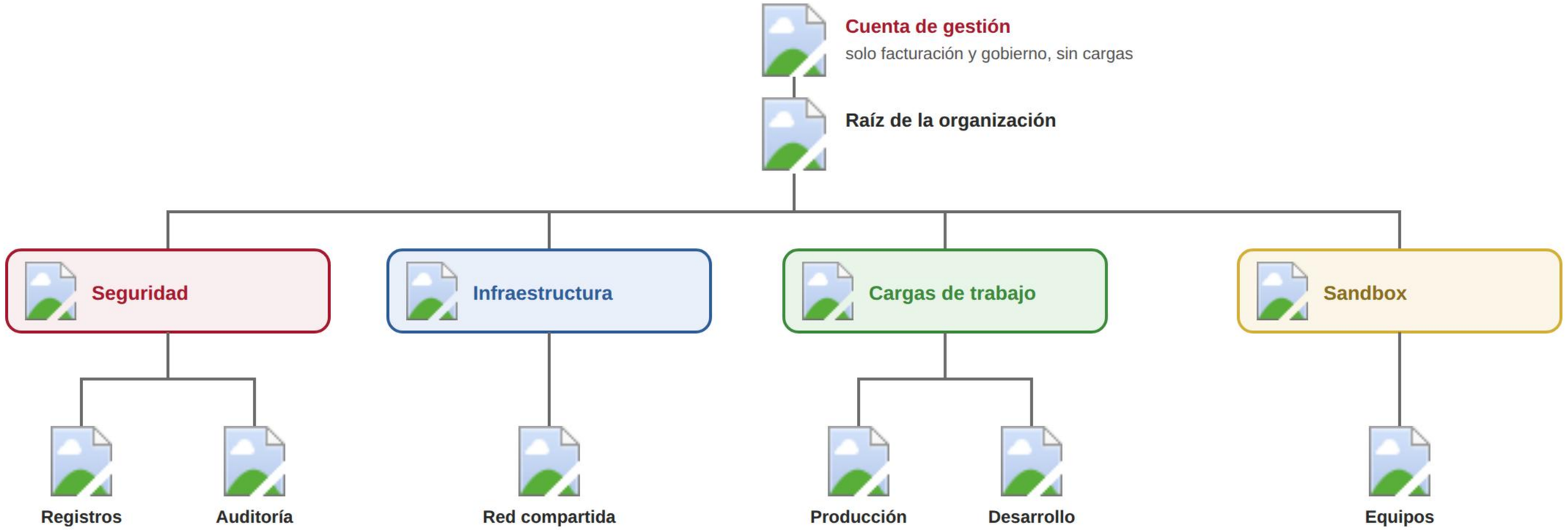
### Cumplimiento
fronteras claras para auditar



*Fuente: adaptado de AWS Whitepaper — Organizing Your AWS Environment Using Multiple Accounts*

# AWS Organizations: la raíz, las unidades y las cuentas

Las cuentas se agrupan en unidades organizativas y heredan políticas desde la raíz.




## Qué da la jerarquía

**Políticas heredadas**
lo aplicado a una unidad llega a sus cuentas

**Facturación consolidada**
una sola factura, detalle por cuenta

**Cuentas nuevas por API**
con línea base de seguridad

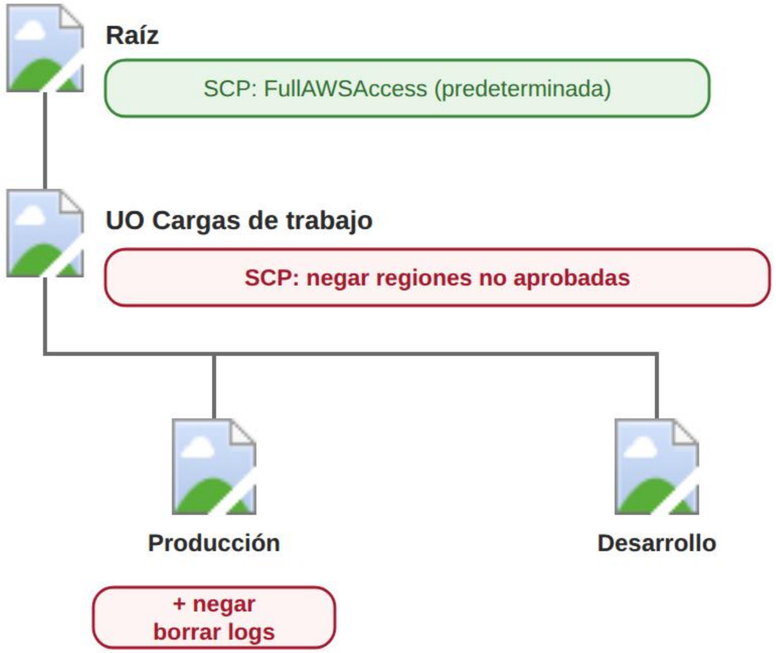
**Servicios compartidos**
compartir recursos entre cuentas



*Fuente: adaptado de AWS Organizations User Guide — Terminology and concepts*

# SCP: el techo de toda la organización

Una SCP no concede permisos: limita lo que cualquier identidad de la cuenta podría llegar a hacer.




### Permiso efectivo en Producción

**= lo que concede IAM ∩ lo que permiten todas las SCP del camino**

La SCP nunca concede: si IAM no da el permiso, no hay permiso aunque la SCP lo deje pasar.

Si cualquier SCP del camino niega o no permite la acción, la solicitud se rechaza.

*Ejemplo: aun siendo administradora, Ana no puede lanzar EC2 en una región no aprobada.*

### Lo que debes recordar

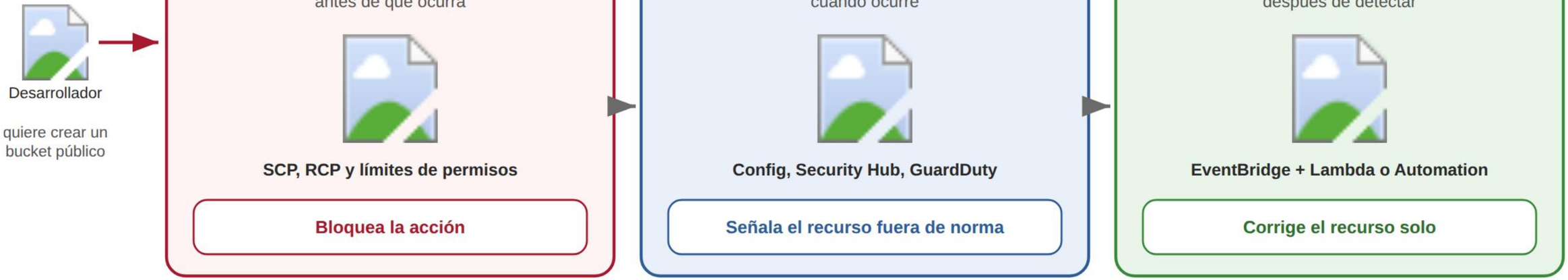
1. **Son un techo, no un permiso**
   Filtran lo que IAM puede conceder.
2. **Heredan hacia abajo**
   Raíz, UO y cuenta suman sus restricciones.
3. **Afectan a toda la cuenta miembro**
   Usuarios, roles y usuario raíz de la cuenta.
4. **No tocan la cuenta de gestión**
   Por eso no se ponen cargas allí.
5. **Ojo con la lista de permitidos**
   Quitar FullAWSAccess sin reemplazo corta todo.



*Fuente: adaptado de AWS Organizations User Guide — Service control policies (SCPs)*

# Guardarraíles: prevenir, detectar y corregir

Lo ideal es impedir; lo realista es impedir, detectar lo que se escape y corregirlo solo.




## Ejemplo con el mismo bucket

**1 Prevenir**

Una SCP niega s3:PutBucketPublicAccessBlock con valor falso y s3:PutBucketPolicy con acceso público: el intento falla.

**2 Detectar**

Si el bucket se creó antes de la regla, una regla de Config lo marca como no conforme y Security Hub lo centraliza.

**3 Corregir**

Un evento de EventBridge lanza una automatización que activa el bloqueo de acceso público y avisa al dueño.



*Fuente: adaptado de AWS Security Reference Architecture y AWS Control Tower — controles*

# Control Tower: una organización lista desde el día uno

El servicio automatiza la estructura, los registros centrales y los controles obligatorios.



Control Tower

**Landing zone: una organización lista con línea base de seguridad**

### Organización creada por Control Tower



Gestión



**UO Seguridad**



Archivo de registros



Auditoría

CloudTrail y Config de toda la organización, protegidos.



**UO Cargas (propia)**



Producción



Desarrollo

Cuentas nuevas desde la fábrica de cuentas.

### Tres tipos de controles

**Preventivos**

SCP: impiden la acción

**Detectivos**

reglas de Config: señalan

**Proactivos**

revisan la plantilla antes de desplegar

### Fábrica de cuentas

**Solicitud**
por el catálogo de servicios

**Cuenta nueva**
dentro de la UO correcta

**Línea base**
registros, roles y red

**Controles**
aplicados desde el día uno



*Fuente: adaptado de AWS Control Tower User Guide — landing zone y controls*

# Azure: grupos de administración y herencia de políticas

La misma idea de jerarquía: lo que se asigna arriba se hereda hasta los recursos.



**Grupo de administración**

CIC-Raíz



**Suscripción**

Producción · Desarrollo



**Grupo de recursos**

rg-pedidos · rg-planta



**Recursos**

máquinas virtuales, bases de datos, discos



**Azure Policy asignada en CIC-Raíz**

Regla: ubicaciones permitidas

```
"allowedLocations": ["brazilsouth",
                     "eastus"]
"effect": "deny"
```

**Se hereda a todo lo que cuelga de ese grupo.**

**Los roles (RBAC) también se heredan**

Asignar Lector al equipo de auditoría en el grupo de administración le da lectura en todas las suscripciones.

**Mismo concepto, otro nombre**

En AWS: unidades organizativas y cuentas.
En Azure: grupos de administración y suscripciones.



*Fuente: adaptado de Microsoft Learn — Azure governance: management groups, Policy y RBAC*

# Azure Policy y bloqueos: reglas y protección

Policy exige la configuración correcta; los bloqueos evitan el borrado accidental.

## Azure Policy: de la definición al cumplimiento

**1 Definición**
la regla: qué se exige

**2 Asignación**
a un ámbito: grupo, suscripción

**3 Evaluación**
al crear, cambiar y de forma periódica

**4 Cumplimiento**
tablero de recursos conformes

### Efectos más usados

**Deny**
rechaza la creación

**Audit**
solo registra el incumplimiento

**Modify / Append**
agrega o corrige etiquetas

**DeployIfNotExists**
despliega lo que falta

*Iniciativa = un conjunto de políticas con un objetivo común, como cumplir una norma de seguridad.*

### Bloqueos de recursos

Protegen incluso del propietario

- **CanNotDelete: se puede leer y editar**
- **ReadOnly: ni cambiar ni borrar**

Se aplican a suscripción, grupo de recursos o recurso.

### Ejemplo: grupo rg-planta




**Eliminar: bloqueado por el bloqueo**

**Para borrar, primero se quita el bloqueo: eso deja huella.**



*Fuente: adaptado de Microsoft Learn — Azure Policy overview y Lock your resources; presentación del docente*

# Etiquetado: la base de saber quién gasta qué

Sin etiquetas coherentes no hay asignación de costos ni FinOps posible.

## Esquema de etiquetas de CIC

**Cada recurso lleva las mismas cuatro claves**

- Proyecto = pedidos
- CentroCosto = CC-2301
- Entorno = prod
- Propietario = ti-planta

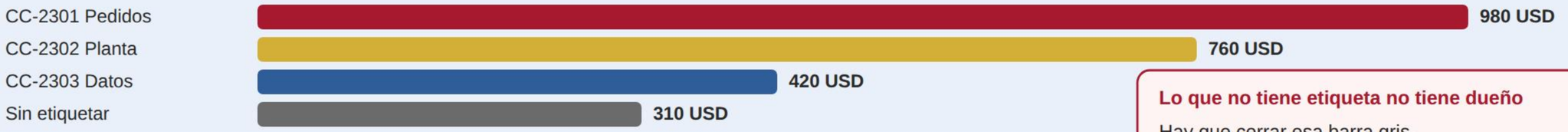
**Política de etiquetas**
fija claves y valores permitidos

**Etiquetas de asignación**
se activan en Facturación

**Cost Explorer**
agrupa el gasto por etiqueta

**Informe por centro de costo**

## Gasto del mes por etiqueta CentroCosto (ilustrativo)




**Lo que no tiene etiqueta no tiene dueño**
Hay que cerrar esa barra gris.

### Gobernar
una política de etiquetas evita Entorno, entorno y ENTORNO

### Activar
las etiquetas de asignación cuentan solo desde su activación

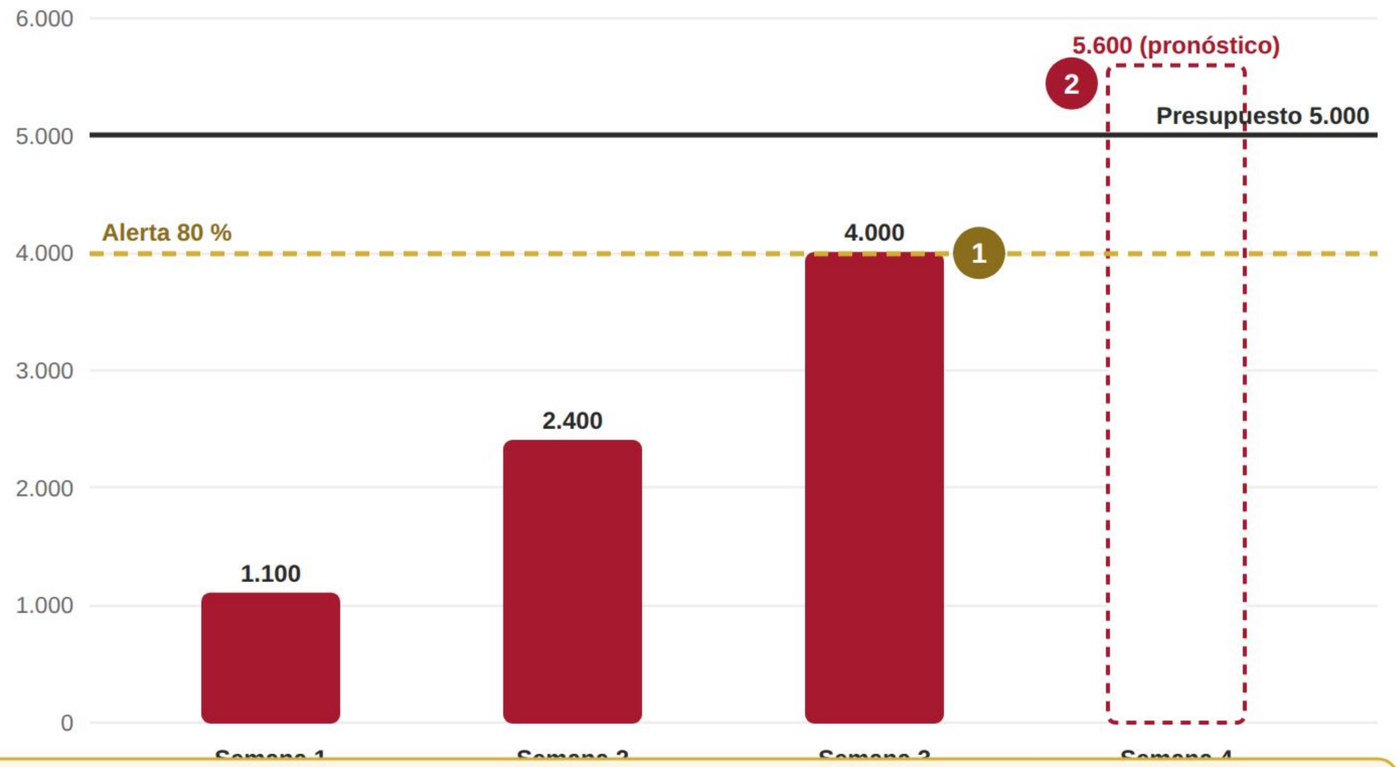
### Reutilizar
las mismas etiquetas sirven para ABAC (Parte 2)


*Fuente: adaptado de AWS Whitepaper — Tagging Best Practices y AWS Billing: cost allocation tags*

# Presupuestos y alertas: enterarte antes de la factura

Una alerta al 80 por ciento y otra por pronóstico dan tiempo para actuar dentro del mes.

**Presupuesto mensual de 5.000 USD, gasto acumulado (ilustrativo)**




**Primero avisa, luego actúa: una alerta sin dueño no sirve.**

Define quién recibe cada umbral y qué decide cuando suene.

## AWS Budgets

umbrales sobre gasto real o pronóstico

1. Gasto real llega al 80 por ciento
2. El pronóstico supera el 100 por ciento

También: por servicio, por etiqueta, por cuenta, y de uso o de Savings Plans.

Correo a finanzas y aviso en el canal del equipo

## Acciones de presupuesto

Al cruzar un umbral, AWS Budgets puede aplicar una política de IAM o una SCP, o detener instancias EC2 y RDS, con tu aprobación o de forma automática.



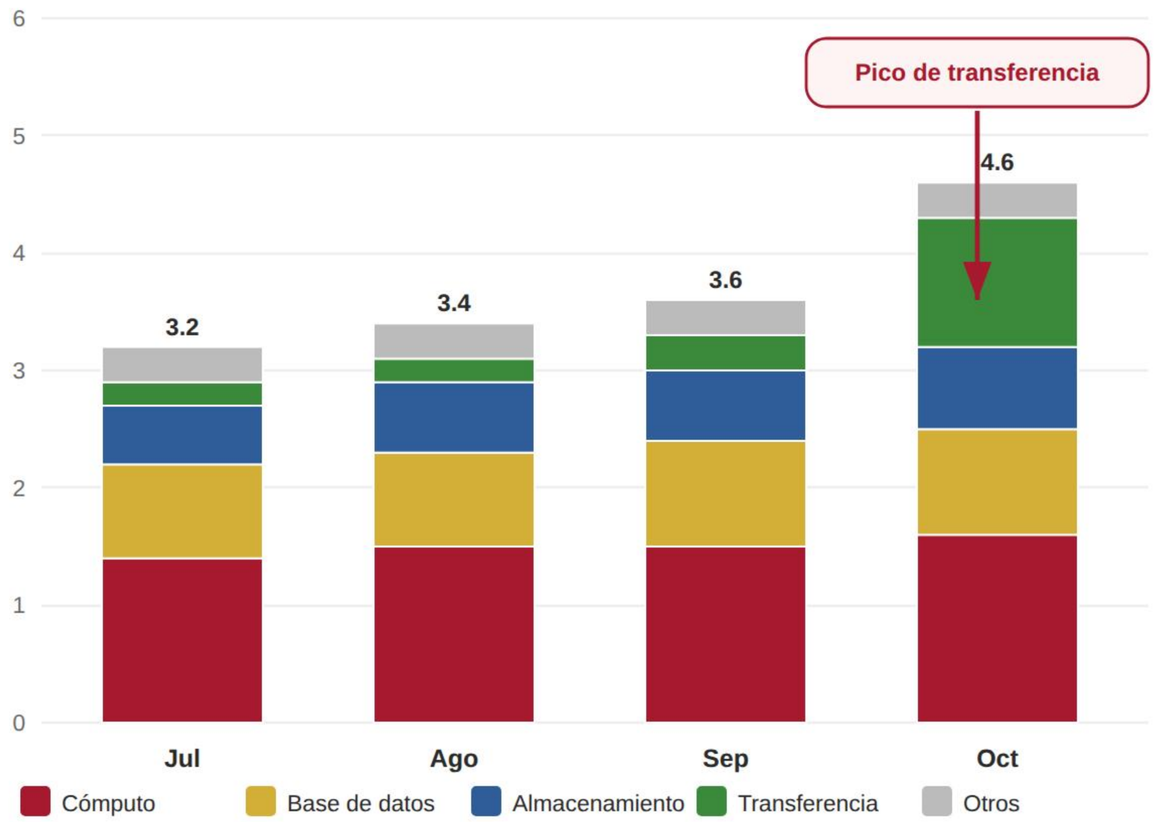
*Fuente: adaptado de AWS Budgets User Guide — Budget actions y notifications; cifras ilustrativas*

COSTOS

# Entender la factura: Cost Explorer e informes

La factura se lee por servicio, cuenta y etiqueta; un pico se investiga el mismo mes.

**Gasto mensual por servicio (miles de USD, ilustrativo)**




### Cost Explorer

Filtra por cuenta, servicio y etiqueta; muestra tendencia y pronóstico.

### Informes de costo y uso

El detalle más fino, entregado en S3 y consultable con Athena.

### Detección de anomalías

Avisa cuando un gasto se sale del patrón habitual.

### Rutina mensual de FinOps

1 Revisar cambios frente al mes anterior
2 Preguntar al dueño de cada pico
3 Ajustar tamaño o apagar lo que sobra



*Fuente: adaptado de AWS Cost Management User Guide — Cost Explorer, Data Exports y Cost Anomaly Detection*

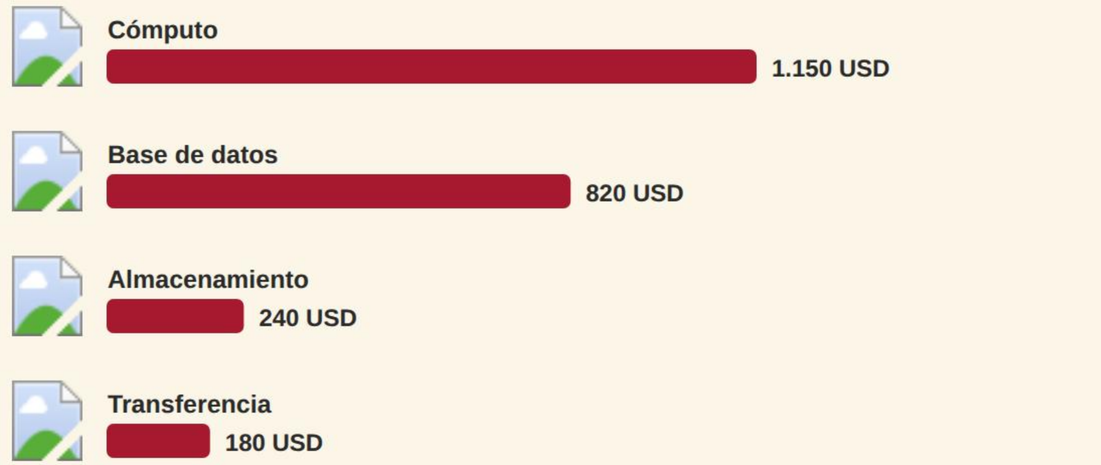
# Calculadora de precios y TCO: estimar y justificar

Una estimación dice cuánto costará una carga nueva; el TCO compara contra seguir en casa.

## Calculadora de precios

Estima una carga nueva antes de construirla

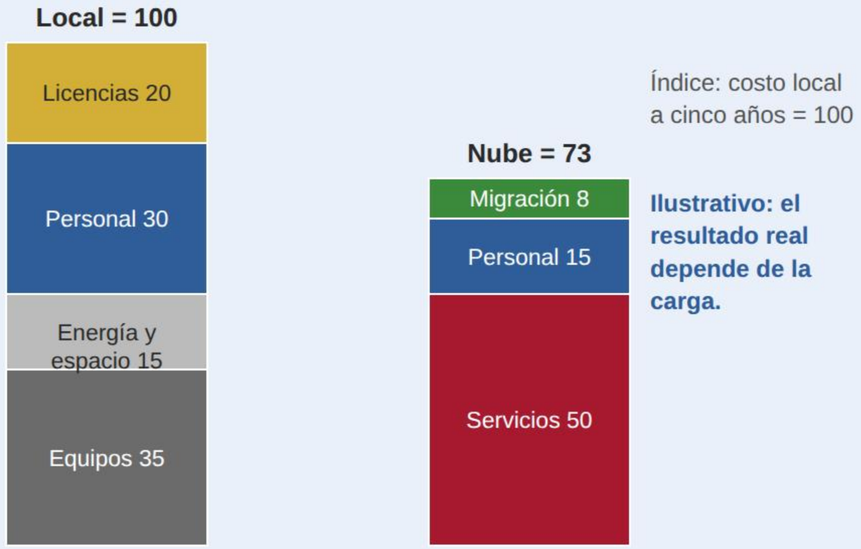
**Portal de pedidos de CIC: estimación mensual**




**Total ilustrativo: ≈ 2.390 USD al mes**

## TCO: costo total de propiedad

Compara a cinco años: local frente a nube




Para estimar usa la calculadora de precios del proveedor; para el caso de negocio de una migración usa herramientas de TCO, como Migration Evaluator en AWS. Incluye costos ocultos: personal, energía, renovación de equipos y tiempo de migración.

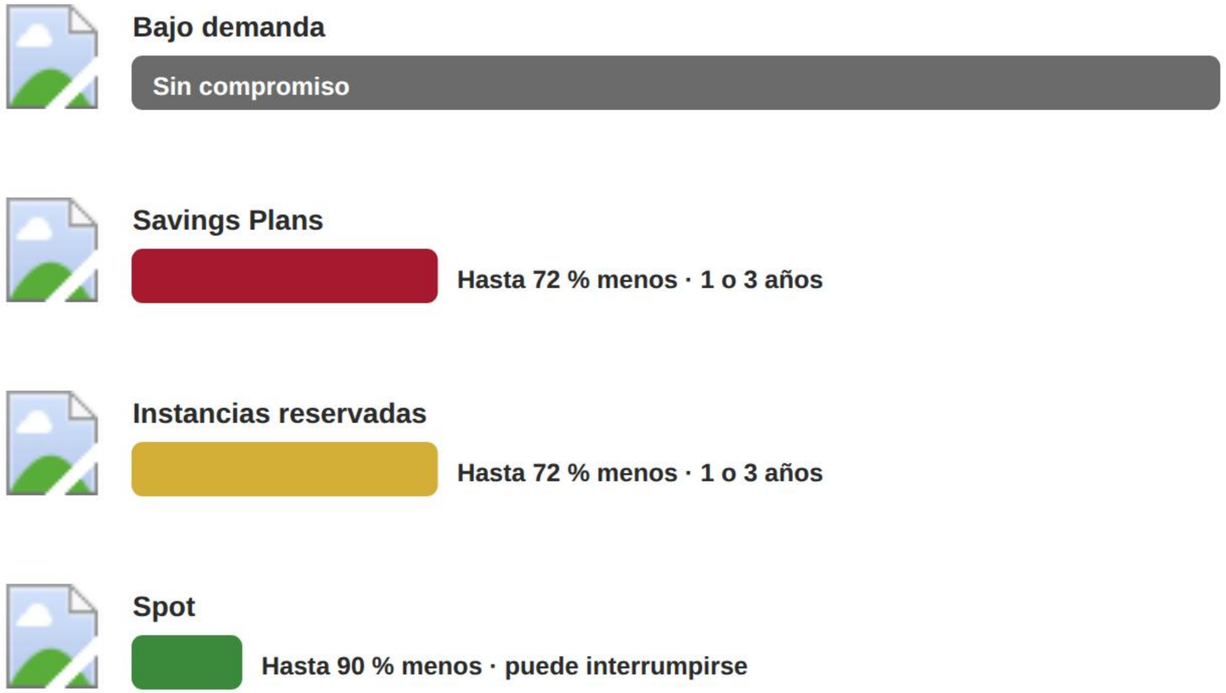


*Fuente: adaptado de AWS Pricing Calculator y AWS Migration Evaluator; cifras ilustrativas*

# Modelos de compra: pagar menos por lo mismo

Cargar la base estable con compromiso y dejar lo variable bajo demanda reduce el costo.

**Precio relativo por hora de cómputo (bajo demanda = 100)**




Los porcentajes son máximos publicados por AWS; el ahorro real depende de la región, el tipo de instancia y el compromiso.

## Cuándo usar cada uno

1. **Bajo demanda**
   picos impredecibles y cargas nuevas
2. **Savings Plans o reservas**
   la base estable que corre todo el año
3. **Spot**
   procesos por lotes que toleran interrupciones
4. **Primero ajustar el tamaño**
   Compute Optimizer detecta instancias sobredimensionadas

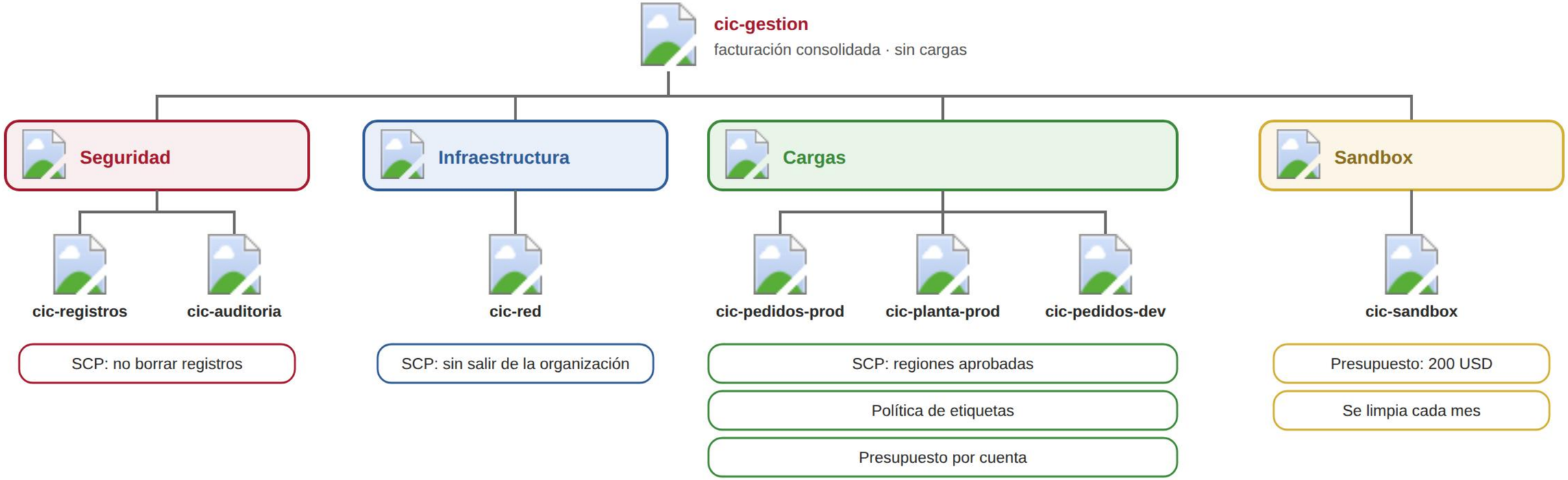


*Fuente: adaptado de AWS Well-Architected, pilar Optimización de costos (COST07)*

CASO INTEGRADOR

# CIC: estructura de cuentas y guardarraíles

Cada cuenta tiene un propósito, un dueño, un presupuesto y los mismos controles de base.




**Reglas que se cumplen en toda la organización**

**Una cuenta, un centro de costo**
finanzas ve el gasto sin hojas de cálculo

**Acceso solo por Identity Center**
sin usuarios IAM ni llaves de largo plazo

**Cuentas nuevas por fábrica**
con registros y controles desde el inicio

**Revisión trimestral**
Access Analyzer y permisos sin uso



*Fuente: adaptado de AWS Whitepaper — Organizing Your AWS Environment; caso transversal CIC*

# Cierre de la Semana 9 y adelanto de la Semana 10

Identidad, permisos, federación y gobierno forman una sola cadena de confianza.

## Semana 9 en cuatro ideas

1



### Credenciales que caducan

Roles, STS y OIDC reemplazan las llaves eternas.

2



### Permisos justos

Condiciones, límites y evaluación en cinco puertas.

3



### Identidad federada

Un directorio, Identity Center, Cognito y secretos con rotación.

4



### Gobierno y costo

Organizations, SCP, etiquetas, presupuestos y TCO.

## Adelanto de la Semana 10 · Elasticidad, alta disponibilidad y monitorización



**Balanceadores**



**Auto Scaling**



**Métricas y alarmas**



**Salud en DNS**

Diseña aplicaciones que crecen, se reparan solas y se observan. Sin laboratorio evaluado esta semana: repasa el módulo 9 de AWS Academy.



*Fuente: resumen de la Semana 9; temario del curso (S10)*